



# 程序设计实践 编程要点

---

机器人路径规划软件开发

串口通信软件设计

网口通信软件设计

# 主要内容

---

- ◆ 对话框的创建与控件使用
- ◆ C++图形绘制
- ◆ 鼠标响应
- ◆ 定时器



# 对话框的创建与控件使用

---



- 对话框是Windows应用程序中一种常用的资源。主要功能是输出信息和接收用户的输入数据。
- 控件是嵌入在对话框中的一个特殊的小窗口，用于完成不同的输入、输出功能。
- 对话框依靠控件与用户进行信息交互
  - 显示提示信息：例如程序启动时显示版权、显示进度信息
  - 接收用户输入的数据



UpdateData(TRUE)

UpdateData(FALSE)

成员变量

对话框对象

更新成员变量

初始化成员变量

- 在MFC中，对话框的功能被封装成**CDialog**类，CDialog类是CWnd类的派生类
- CDialog类的派生关系如下所示：

**CObject**

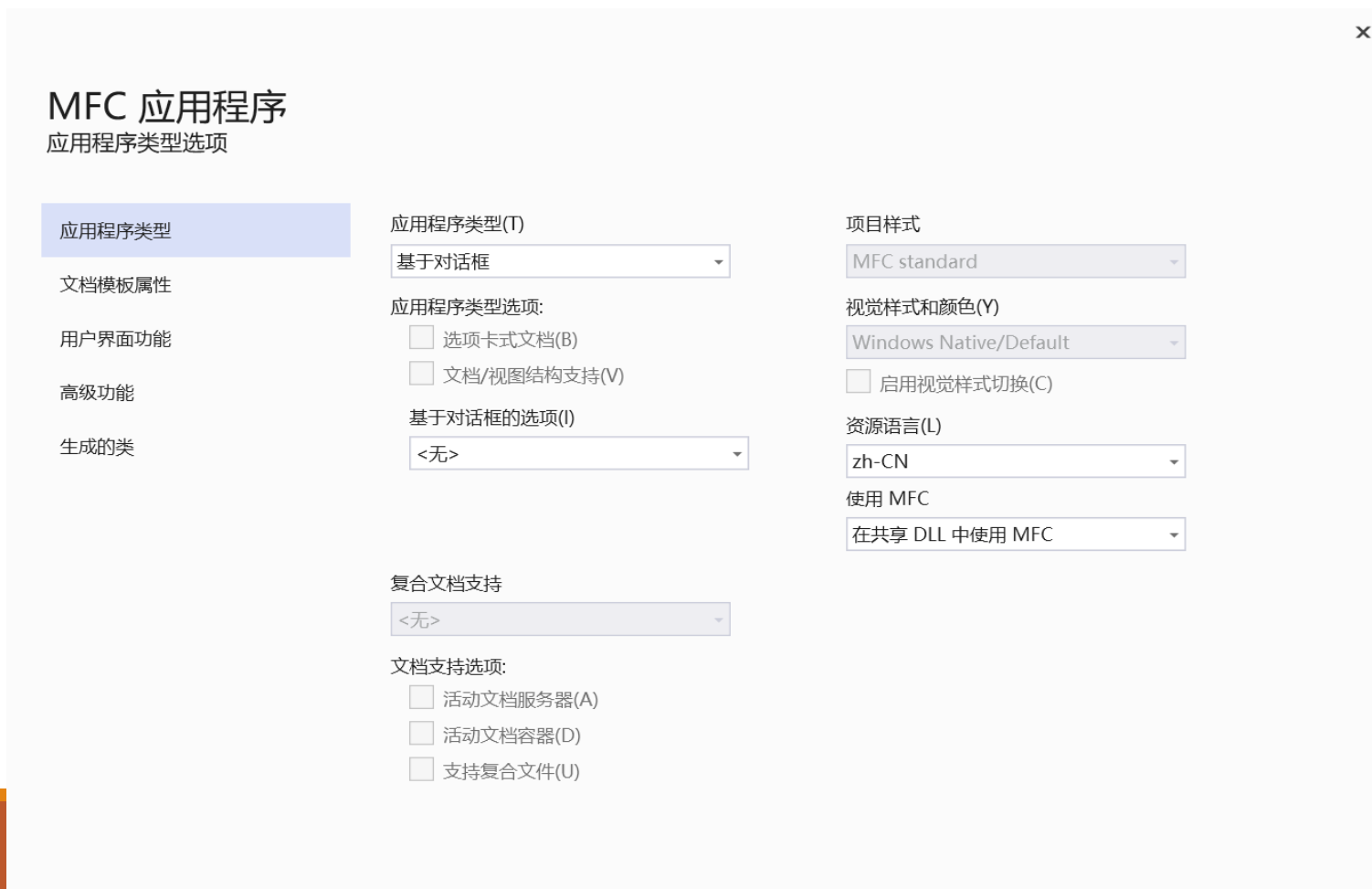
**CCmdTarget**

**CWnd**

**Cdialog**

- 在程序中创建的对话框类是CDialog类的派生类

- 对话框可以作为一个程序的主窗口：基于对话框的应用程序
- 对话框可以通过菜单命令或工具栏按钮打开



MFC 应用程序  
应用程序类型选项

应用程序类型

文档模板属性

用户界面功能

高级功能

生成的类

应用程序类型(T)

基于对话框

应用程序类型选项:

☐ 选项卡式文档(B)

☐ 文档/视图结构支持(V)

基于对话框的选项(I)

<无>

复合文档支持

<无>

文档支持选项:

☐ 活动文档服务器(A)

☐ 活动文档容器(D)

☐ 支持复合文件(U)

项目样式

MFC standard

视觉样式和颜色(Y)

Windows Native/Default

☐ 启用视觉样式切换(C)

资源语言(L)

zh-CN

使用 MFC

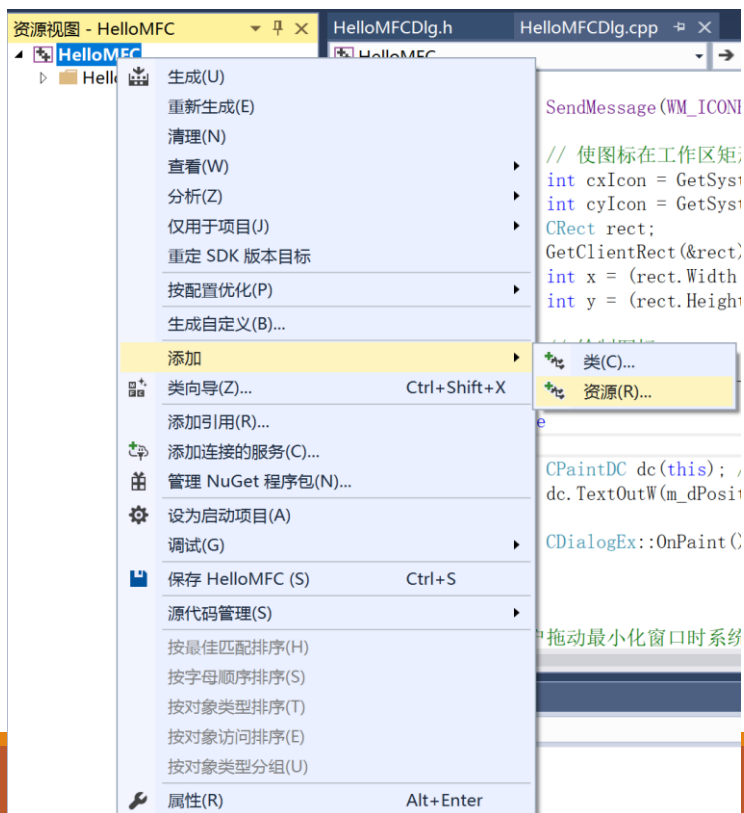
在共享 DLL 中使用 MFC

## 对话框创建和使用：

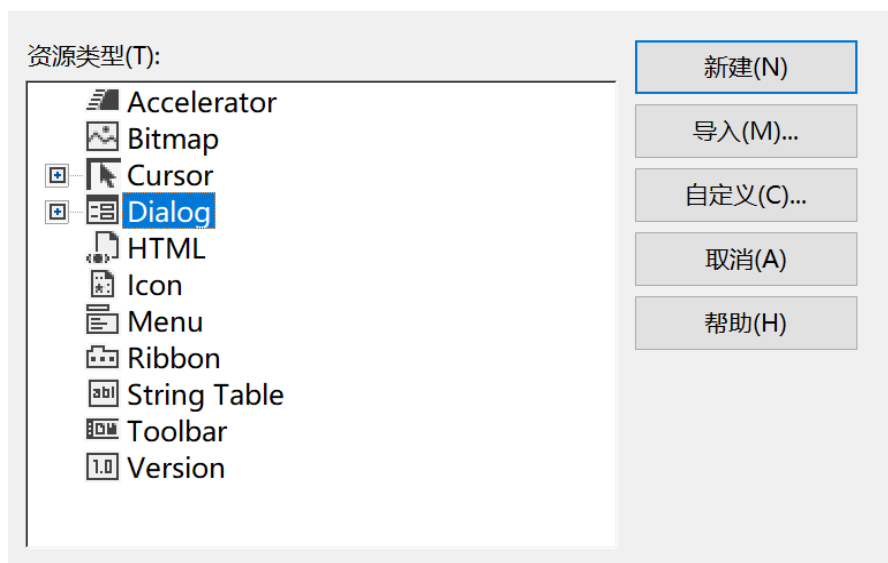
- ① 首先创建对话框资源：指定对话框本身的属性（如大小、位置、风格、类型等），添加控件并设置控件属性；
- ② 再创建一个基于该对话框资源的对话框类：规定对话框和对话框中每个控件的行为（编写消息响应函数）
- ③ 包含对话框头文件，声明对话框类对象并使用

# 1、添加对话框资源

从菜单项目|添加资源命令，弹出Insert Resource（添加资源）资源列表框。在Insert Resource框中直接选择Dialog项，然后单击New按钮。。

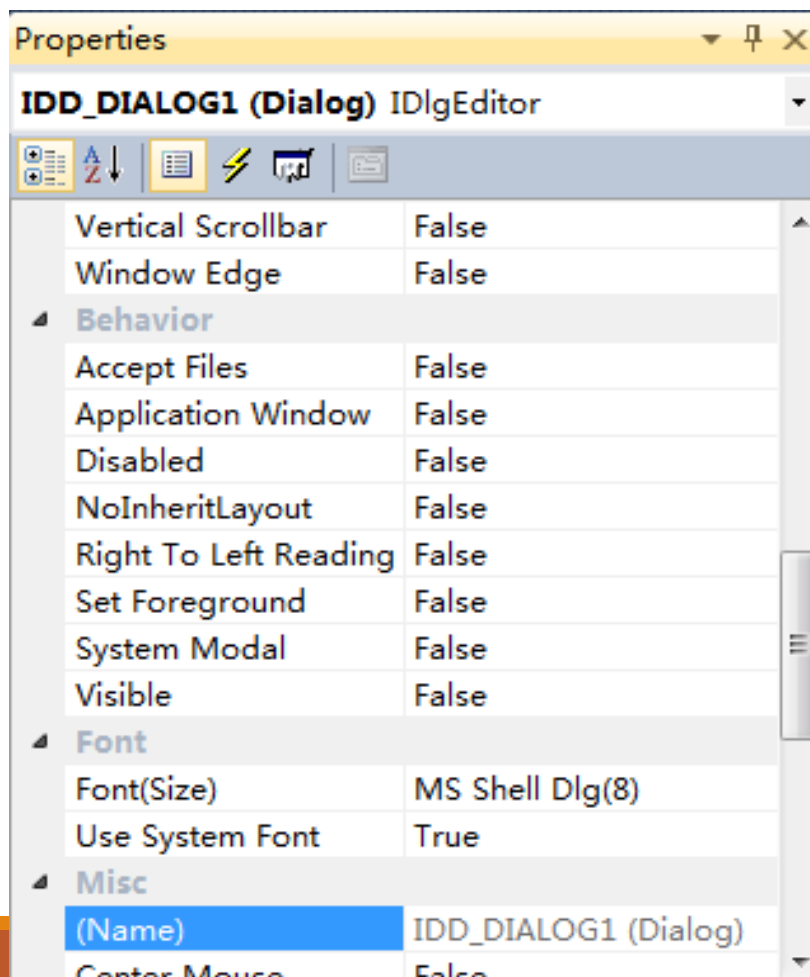


添加资源



## 2、对话框属性编辑

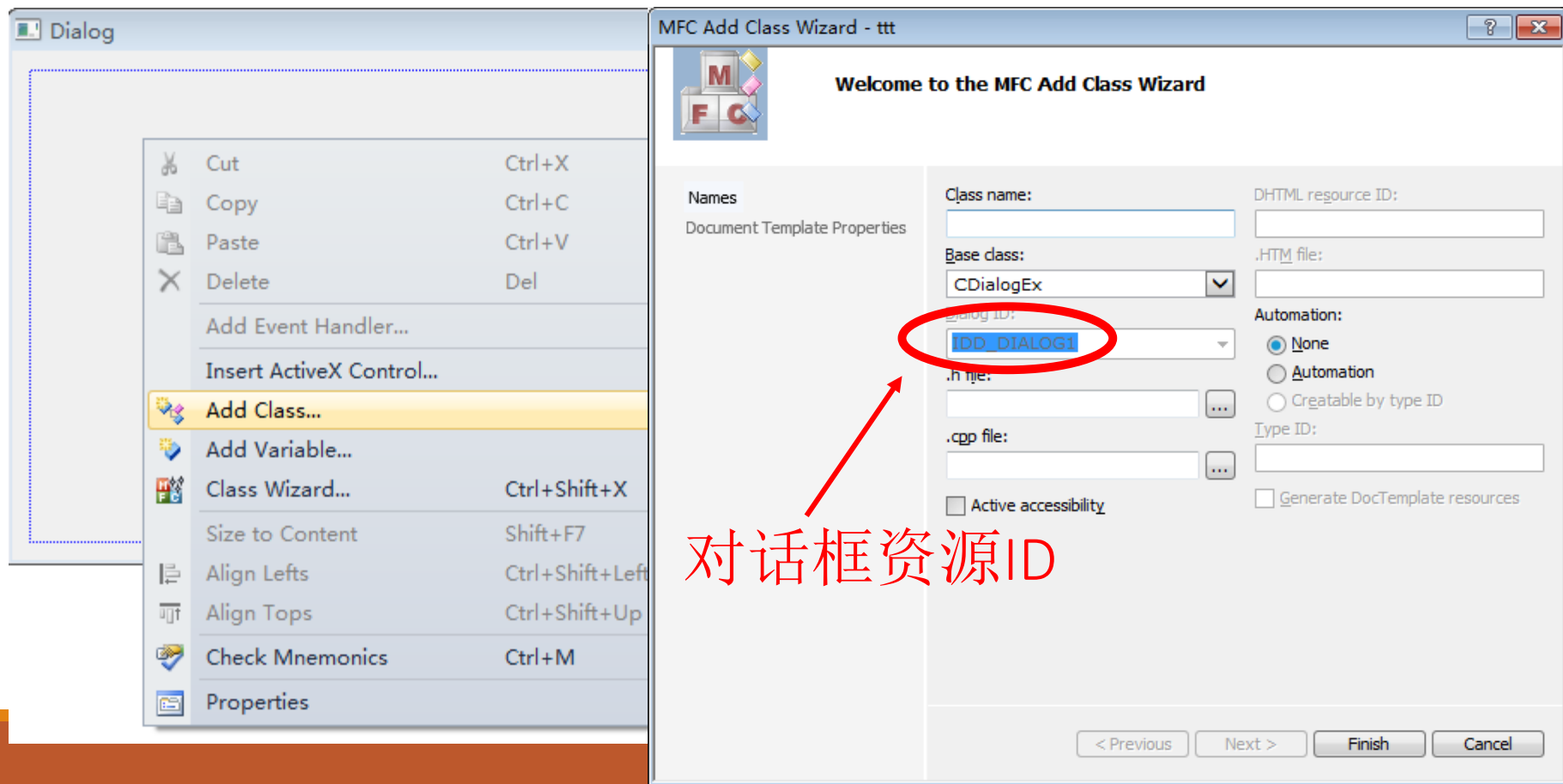
在对话框资源的非控件区域点击右键在弹出的快捷菜单中选择Properties，在弹出的信息栏中修改对话框属性



### 3、创建对话框类：

在对话框资源的非控件区域双击鼠标，或者点击右键在弹出的快捷菜单中选择Add Class，将弹出类添加向导对话框

在类向导中输入类名，点击Finish按钮完成类的添加



# 对话框的常用成员函数

## CDialog::CDialog()

通过调用派生类构造函数，根据对话框资源模板定义一个对话框。

## CDialog::DoModal()

激活模态对话框，显示对话框窗口。

## CDialog::Create()

根据对话框资源模板创建非模态对话框窗口。如果对话框不是Visible属性，还需通过调用CWnd::ShowWindow()函数显示非模态对话框窗口。

## CDialog::OnOk()

单击OK按钮时调用该函数，接收对话框输入数据，关闭对话框。

## CDialog::OnCancel()

单击Cancel按钮或按Esc键时调用该函数，不接收对话框输入数据，关闭对话框。

## CDialog::OnInitDialog()

**WM\_INITDIALOG**消息处理函数，在调用DoModal或Create函数时系统发送WM\_INITDIALOG消息，在显示对话框前调用该函数进行初始化。

**CDialog::EndDialog()**：用于关闭模态对话框窗口。

## CWnd::ShowWindow()

显示或隐藏对话框窗口

## CWnd::DestroyWindow()

关闭并销毁非模态对话框

## CWnd::UpdateData()

通过调用DoDataExchange()设置或获取对话框控件的数据

**CWnd::DoDataExchange()**：被UpdateData()调用以实现对话框数据交换，不能直接调用。

**CWnd::GetWindowText()**

获取对话框窗口的标题

**CWnd::SetWindowText()**

修改对话框窗口的标题

**CWnd::GetDlgItemText()**

获取对话框中控件的文本内容

**CWnd::SetDlgItemText()**

设置对话框中控件的文本内容

**CWnd::GetDlgItem()**

获取控件或子窗口的指针

**CWnd::MoveWindow()**

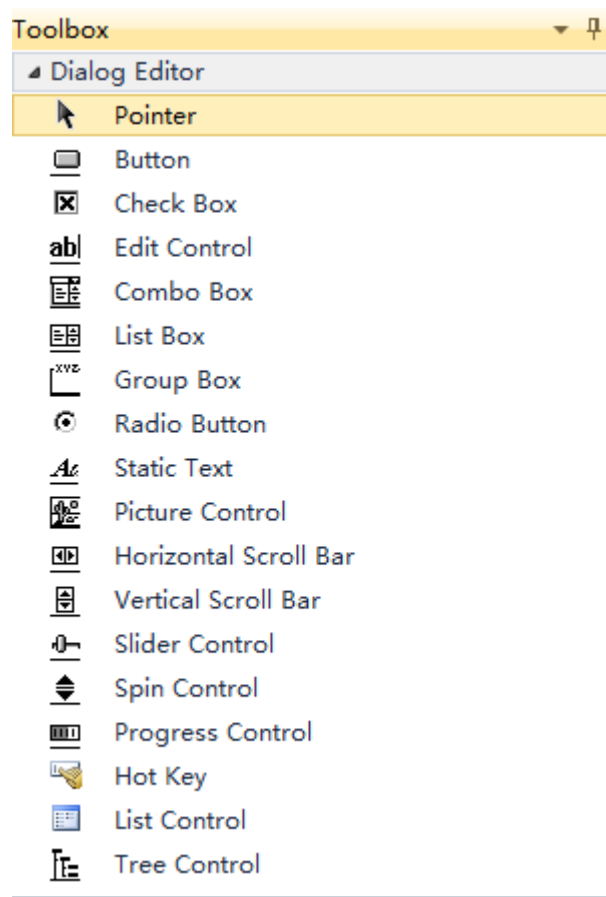
用于移动对话框窗口

**CWnd::EnableWindow()**

使窗口处于禁用或可用状态

## 4、为对话框添加控件：

通过Toolbox信息栏为对话框添加控件



## 5、添加与控件关联的成员变量

在控件上右键点击，在弹出快捷菜单中选择  
Add Variable...

Add Member Variable Wizard - ttt

Welcome to the Add Member Variable Wizard

Access: public

Control variable: ☒

Variable type: CEdit

Control ID: IDC\_EDIT2

Category: Control

Control type: EDIT

Max chars:

Min value:

Max value:

.h file:

.cpp file:

Comment (// notation not required):

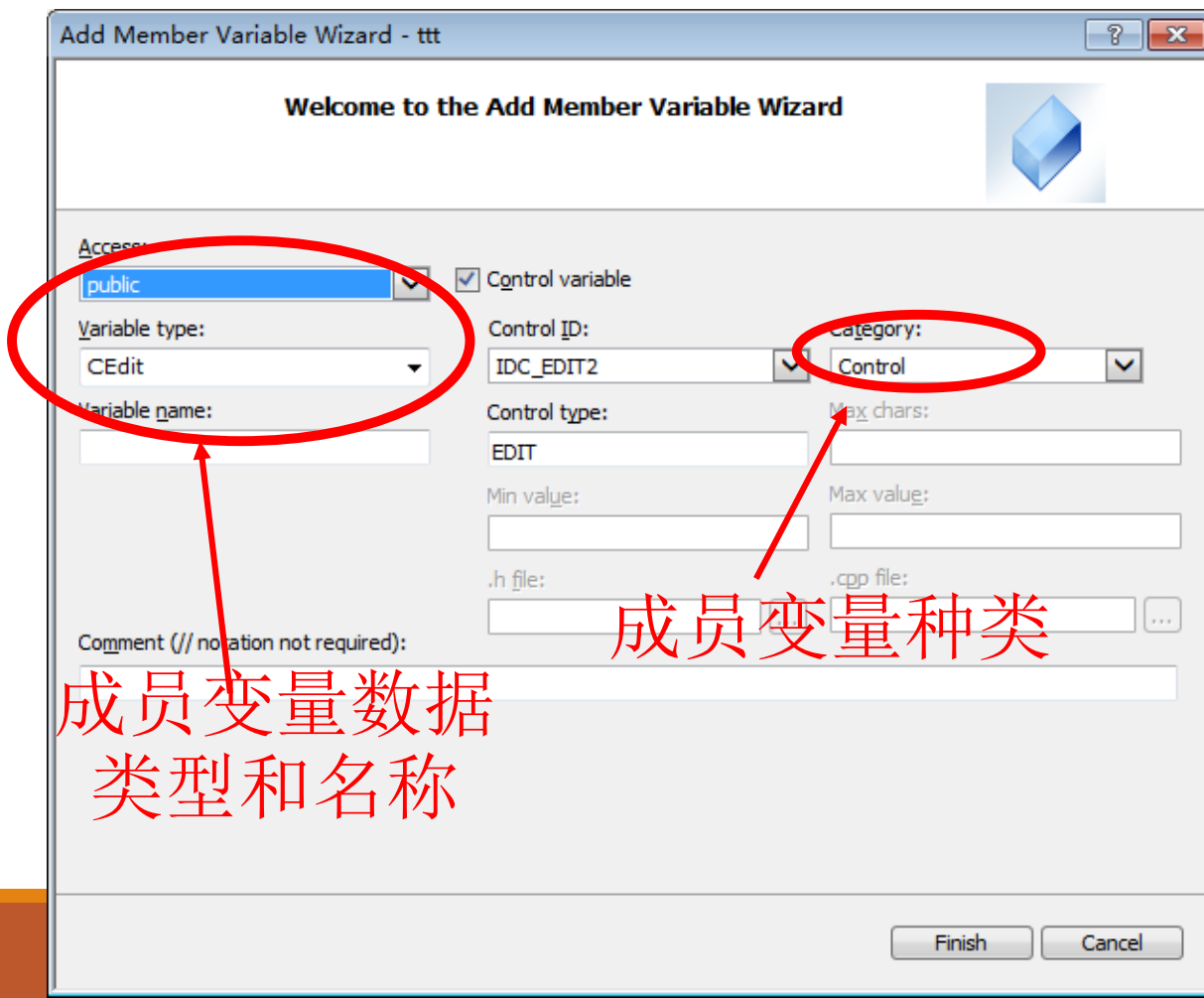
Finish Cancel

**注意：**

只有为对话框资源添加了对话框类之后，才能为控件添加关联的成员变量

## 5、添加与控件关联的成员变量

在控件上右键点击，在弹出快捷菜单中选择Add Variable...



## 5、添加与控件关联的成员变量

### 成员变量种类

- Control: 关联的成员变量是一个MFC控件类对象，可以通过该对象访问控件类的成员变量和调用控件类的成员函数，实现对控件行为的控制和管理；
- Value: 为该控件的某项属性关联一个成员变量，这意味着程序关心的是控件中的值，而不是控件本身。这时，需要指定成员变量数据类型（一般的C++数据类型或Visual C++自定义的数据类型）

## 5、添加与控件关联的成员变量

### 成员变量种类

- Control: 为与控件关联的变量。该对象访问该控件时，实现对控件的操作。
- Value: 为与控件无关的变量。程序关心该变量的值，并据此确定成员变量的值。该变量由程序员定义的数

### 注意:

- 可以为一个控件同时定义一个Control类别的变量和一个Value类别的变量
- 这些变量都是作为对话框派生类的成员变量
- 为了在程序其它地方能够直接访问添加的成员变量，它们都被声明为public属性。

# 常用的MFC控件和控件类 (I)

| MFC类             | 控 件            |
|------------------|----------------|
| <b>CStatic</b>   | 静态文本、图片控件      |
| <b>CEdit</b>     | 编辑框            |
| <b>CButton</b>   | 按钮、复选框、单选按钮、组框 |
| <b>CComboBox</b> | 组合框            |
| <b>CListBox</b>  | 列表框            |
| CScrollBar       | 滚动条            |
| CSpinButtonCtrl  | 旋转按钮           |
| CProgressCtrl    | 进度条            |
| CSliderCtrl      | 滑块             |
| <b>CListCtrl</b> | 列表视控件          |

## 常用的MFC控件和控件类 (II)

| MFC类           | 控 件     |
|----------------|---------|
| CTreeCtrl      | 树视控件    |
| CTabCtrl       | 标签      |
| CAnimateCtrl   | 动画控件    |
| CRichEditCtrl  | 复合编辑框   |
| CDateTimeCtrl  | 日期时间选取器 |
| CMonthCalCtrl  | 日历      |
| CComboBoxEx    | 扩展组合框   |
| CStatusBarCtrl | 状态条控件   |
| CToolBarCtrl   | 工具条控件   |
| CImageList     | 图像列表    |

对于控件Value类型的成员变量，可以通过对话框数据交换DDX和验证DDV实现数据输入输出

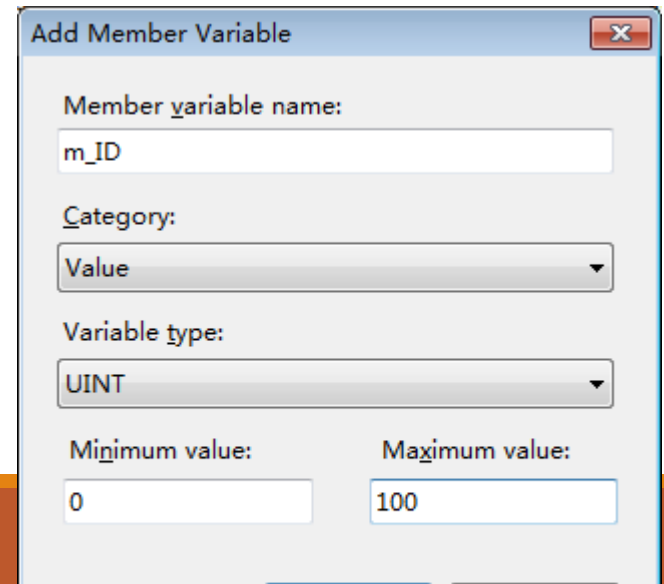
- 对话框数据验证DDV（Dialog Data Validation）：对用户输入数据的有效性进行验证
- 对话框数据交换DDX（Dialog Data Exchange）：实现控件与成员变量间的数据交换

## DDV使用方法:

把对话框中一个编辑框和UINT型的成员变量m\_nInput时指定它的最小值和最大值分别为0和100，则ClassWizard类向导将在成员函数DoDataExchange()中添加以下DDV函数的调用语句：

**DDV\_MinMaxUInt(pDX, m\_nInput, 0, 100);**

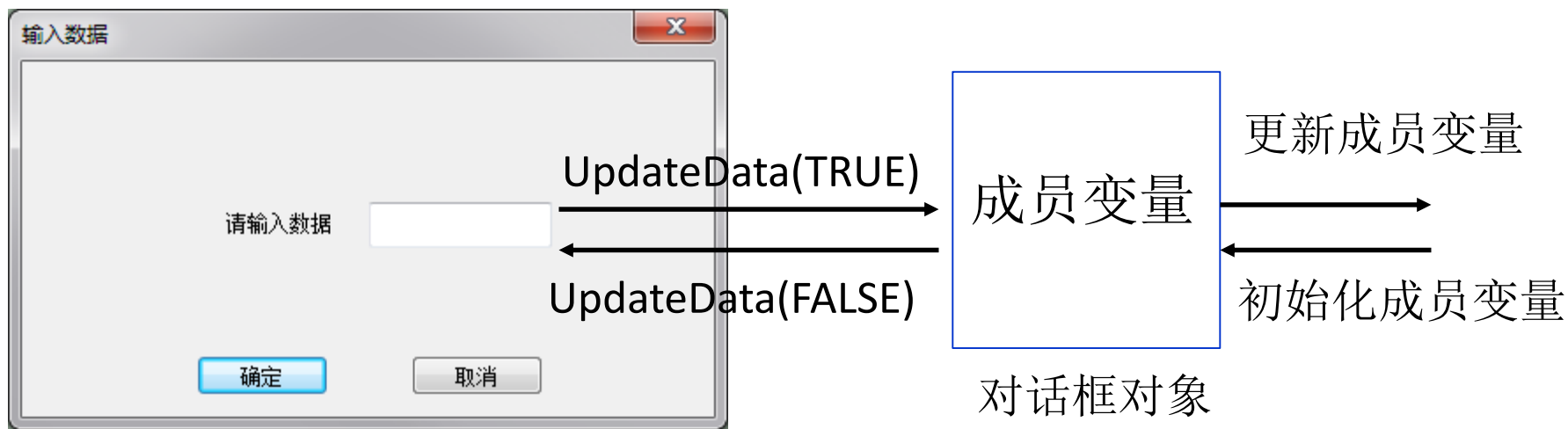
程序运行时，如果用户的输入数据超出**0~100**的范围，**DDV**将显示一个**提示信息对话框**，提示用户有效的输入范围。



## DDX使用方法:

调用**UpdateData(TRUE)**时，MFC通过调用DDX函数将数据从控件传递到关联的成员变量；

调用**UpdateData(FALSE)**时，MFC通过调用DDX函数将数据从成员变量传递到关联的的控件。



## 注意：

创建并显示对话框时，将自动调用 **UpdateData(FALSE)**，将数据从成员变量传递到关联的控件，从而显示在控件中。

单击OK按钮将调用 **CDialog::OnOk()** 函数，在 **OnOk()** 中调用了 **UpdateData(TRUE)**，将数据从控件传递到关联的成员变量。

DDX就如同一条双向通道，而方向控制开关就是 **UpdateData()** 函数中的 **BOOL** 类型参数是 **TRUE** 还是 **FALSE**。

## 6、使用对话框

声明对话框类对象（注意：包含对话框类头文件）

调用对话框类的DoModal()成员函数创建并显示模态对话框

无模态对话框的使用请参考相关资料

# 函数OnOK()、 OnCancel()与DoModal()的关系:

对话框中一般都有ID为IDOK和IDCancel的按钮

单击ID为IDOK的按钮时调用CDialog::OnOK()函数

单击ID为IDCancel的按钮时调用CDialog::OnCancel()函数。

OnOK()函数和OnCancel()函数都将结束DoModal()函数调用，但令DoModal()返回不同的值。

- OnOK()函数令DoModal()返回IDOK，
- OnCancel()函数令DoModal()返回IDCANCAL。
- 用户可以根据DoModal()的返回值做出不同的选择。如：

```
if (myDlg.DoModal() == IDOK )  
{  
    .....  
}
```

## 对话框的初始化:

可以在三个不同的阶段所调用的函数中进行:

- (1) 对话框类构造函数;
- (2) WM\_CREATE消息处理函数;
- (3) WM\_INITDIALOG消息处理函数。

- 建议在消息WM\_INITDIALOG的消息处理函数OnInitDialog()中进行初始化
  - 在收到WM\_INIT\_DIALOG消息时, 对话框的框架已创建, 对话框中的每个控件也已建立起来, 但它们还没有在屏幕上显示
  - 此时自然可以设置或优化对话框中各个控件的外观、尺寸、位置以及其它属性
  - 函数OnInitDialog()将被成员函数DoModal()调用。

## 7、基于对话框的应用程序

可以直接利用MFC AppWizard向导创建一个基于对话框的应用程序；

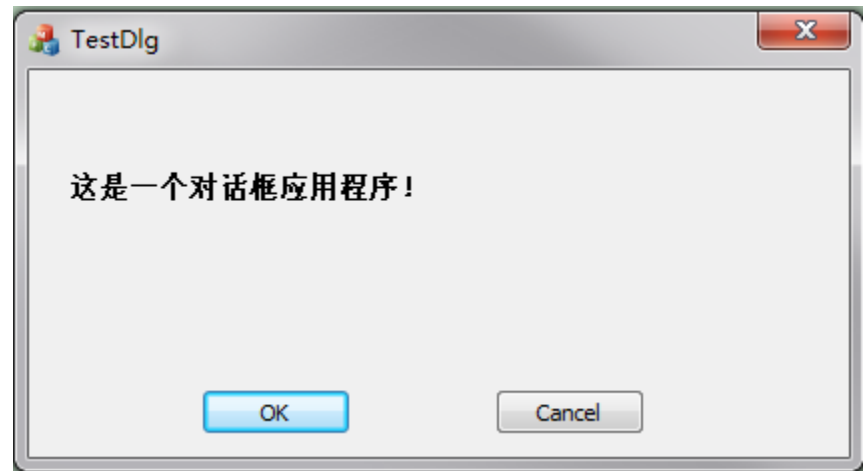
向导将自动完成：

- 添加对话框资源
- 为对话框资源添加对话框类，用对话框类声明对象
- 通过对话框类的DoModal()创建并显示对话框，该对话框即为程序的主窗口

程序编写：添加控件，添加成员变量，添加消息响应函数

1. 执行File|New|Project命令出现New Project对话框，选择MFC|MFC Application项，输入程序名MyDialog，单击OK按钮。在随后出现的Application type对话框窗口中选择Dialog Based选项，单击Finish按钮就创建了应用程序项目，并在Developer Studio中打开了对话框编辑器和控件工具栏。
2. 去掉对话框中标题为“TODO: 在这里设置对话控制”的静态文本控件，调整对话框大小，在成员函数CMyDialogDlg::OnPaint()中添加代码。

```
void CMyDialogDlg::OnPaint()
{
    . . . . .
    else
    {
        CPaintDC dc(this);
        dc.SetBkMode(TRANSPARENT);
        dc.TextOut(20, 50, "这是一个对话框应用程序！");
        CDialog::OnPaint();
    }
}
```



## 8、提示信息对话框

提示信息对话框也称消息对话框，用来显示有关的提示信息。提示信息对话框是一种最简单的对话框，不需要用户自己创建它就可以直接使用，Visual C++提供了相应的函数实现这样的功能：

MFC全局函数，可以在程序中任何地方使用：

```
int AfxMessageBox(LPCTSTR lpText, UINT nType=MB_OK,  
    UINT nIDHelp=0);
```

API函数，可以在程序中任何地方使用：

```
int MessageBox(HWND hWnd, LPCTSTR lpText, LPCTSTR lpCaption,  
    UINT nType);
```

CWnd类的成员函数，只能用于控件、对话框、窗口等一些窗口类的成员函数中：

```
int CWnd::MessageBox(LPCTSTR lpText, LPCTSTR lpCaption=NULL,  
    UINT nType=MB_OK);
```

例：在程序中弹出如下图所示的**警告提示信息对话框**并根据用户选择进行相应的处理：

编写的代码如下：

```
int nChoice=MessageBox(_T("文件复制失败！"),_T("错误"),
    MB_ICONWARNING | MB_ABORTRETRYIGNORE);
switch(nChoice)
{
    case IDABORT:    // 用户按下"终止"按钮
        . . . . .
    case IDRETRY:    // 用户按下"重试"按钮
        . . . . .
    case IDIGNORE:   // 用户按下"忽略"按钮
        . . . . .
}
```



## 9、标准控件

Windows提供的控件分为两类：标准控件和公共控件。

标准控件：静态控件、编辑框、按钮、列表框、组合框和滚动条等。利用标准控件可满足大部分用户界面程序设计的要求。

公共控件：滑块、进度条、列表视控件、树视控件和标签控件等，利用公共控件实现应用程序用户界面风格的多样性。

用户对控件操作时Windows产生**控件**消息并发送给父窗口（如对话框）。

标准控件发送**WM\_COMMAND**控件通知消息，公共控件一般发送**WM\_NOTIFY**控件通知消息，或WM\_COMMAND消息。

通过WM\_COMMAND消息参数**标识**发出消息的控件和具体的事件，消息参数中包含了控件的ID标识和通知码（区分消息种类，如BN\_CLICKED单击按钮事件）

程序员不必关心消息具体的发送和接收，只需为控件消息添加响应函数，然后在响应函数中编写具体的处理代码。

# 控件的组织

添加或删除控件

设置控件属性

调整控件的大小

同时选取多个控件：鼠标框选，shift或ctrl+鼠标点选

复制控件：复制粘贴，ctrl+拖动控件

编排控件：使用主菜单format下的各个子菜单

测试对话框和控件的方法：按下Ctrl+T组合键就能测试对话框运行时的界面效果。

# 控件的公共属性

| 项 目      | 说 明                            |
|----------|--------------------------------|
| ID       | 控件的标识，对话框编辑器会为每一个加入的控件分配一个默认ID |
| Caption  | 控件的标题，作为程序执行时在控件位置上显示的文本       |
| Visible  | 指明显示对话框时该控件是否可见                |
| Group    | 用于指定一个控件组中的第一个控件               |
| HelpID   | 表示为该控件建立一个上下文相关的帮助标识ID         |
| Disabled | 指定控件初始化时是否禁用                   |
| Tab Stop | 表示对话框运行后该控件可以通过使用Tab键来获取焦点     |

## 1) 静态控件

静态控件（Static control）是用来显示一个文本串或图形信息的控件，它包括静态文本控件、图片控件和组框。

- 静态文本控件用来显示一般不需要变化的文本；
- 图片控件用来显示边框、矩形、图标或位图等图形；
- 组框用来显示一个文本标题和一个矩形边框，通常用来将一组控件组织在一起。

管理静态文本控件和图片控件的MFC类是CStatic类，而管理组框的MFC类是CButton类。

所有静态控件默认的ID标识都为IDC\_STATIC，如果要为一个静态控件添加成员变量或消息处理函数，必须重新为它指定一个唯一的ID标识。

编程时用的最多是静态文本控件，它被用来作为其它控件的标题。每一个静态文本控件最多可以显示255个字符，可以使用“\n”换行符。



## 2) 编辑框

编辑框 (Edit box) 又称文本框或编辑控件，它一般与静态文本控件一起使用，用于数据的输入和输出。编辑框提供了完整的键盘输入和编辑功能，可以输入各种文本、数字或者口令。

编辑框为用户提供了良好的输入、输出功能，能够将键盘输入的字符串转化为要求的数据类型 (DDX)，并验证它是否符合输入要求 (DDV)。

## 编辑框常用属性：

- Multiline 设置单行或多行编辑
- Align text 设置文本对齐方式
- Number 表示只能输入数字
- Password 表示键入编辑框的字符都将显示为 “\*”
- Uppercase 或 Lowercase 表示键入编辑框的字符全部转换成大写或小写形式
- Read-Only 表示只能输出数据。

当编辑框的文本被修改，会向父窗口发送消息，可以利用类向导在对话框类中添加消息处理函数。编辑框发送消息有：

- EN\_CHANGE
- EN\_KILLFOCUS
- EN\_SETFOCUS
- EN\_MAXTEXT
- EN\_UPDATE

# 例 编写基于对话框的程序，设置圆的半径，完成圆形的绘制。

新建基于对话框的应用程序，为对话框资源添加控件：

添加一个静态文本控件，其Caption为“请输入半径”。添加一个Edit Box编辑框控件，其ID标识为IDC\_EDIT\_RADIUS，该控件用于接收用户的输入的半径数据。

添加与控件关联的成员变量m\_nRadius，其数据类型为UINT。使用对话框数据验证DDV功能：m\_nRadius的最小值5和最大值250。

1. 在对话框类的OnInitDialog()中添加下面代码:

```
m_nRadius = 100; //初始化半径输入控件  
UpdateData(FALSE);
```

2. 在对话框类的OnPaint()中添加如下画圆的语句:

```
CPaintDC dc(this); // 用于绘制的设备上下文  
dc.Ellipse(0,0,m_nRadius, m_nRadius);
```

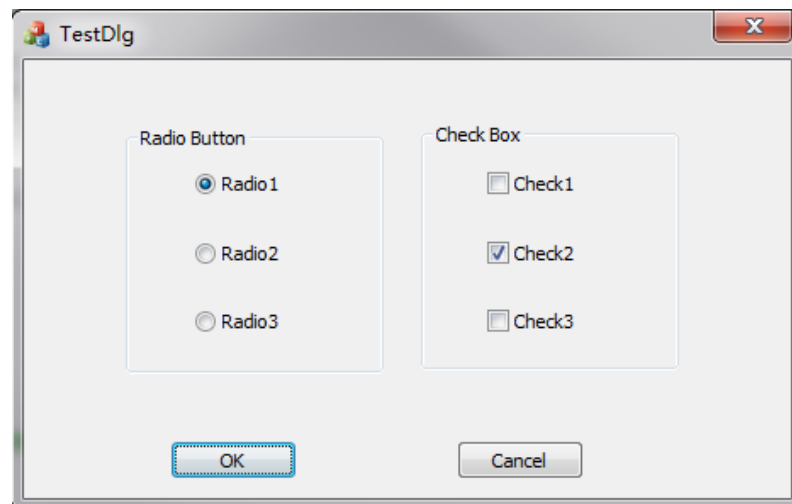
3. 为IDC\_EDIT\_RADIUS控件添加EN\_CHANGE消息处理函数，并添加代码:

```
UpdateData(TRUE); //更新控件数据到变量  
Invalidate(); //刷新绘图
```

### 3) 按钮

按钮（Button）包括**按键按钮**、**单选按钮**和**复选框**等3种类型，由MFC的CButton类封装，

- 按键按钮用来执行某个命令，也被称为命令按钮；
- 单选按钮用于在一组互相排斥的选项中选择其中一项；
- 复选框用于在一组选项中选择其中一项或多项；



## 按键按钮

几乎所有的对话框都使用简单的**按键按钮**，如OK按钮。

通过设置Default button属性可将按钮设置成默认按钮。对话框显示时按下Enter键将执行该按钮的命令功能。一个对话框只能有一个Default button默认按钮，通常情况下将OK按钮设置为默认按钮。

其它属性：

- Owner draw利用WM\_DRAWITEM消息处理函数定制按钮的外观，
- Icon表示用图标代替原来的文本标题，
- Bitmap表示用位图代替原来的文本标题，
- Client edge、Static edge、Modal frame用于设置按钮的外观。

## 单选按钮

- ✓ 一般将一组单选按钮放在一个组框中
- ✓ 在一组单选按钮中，**第一个 (Tab键顺序) 按钮最重要**，其ID值可用于在对话框中为控件建立关联的成员变量
- ✓ **必须为同组中的第一个单选按钮设置Group属性**，而同组的其它单选按钮不可再设置Group属性。

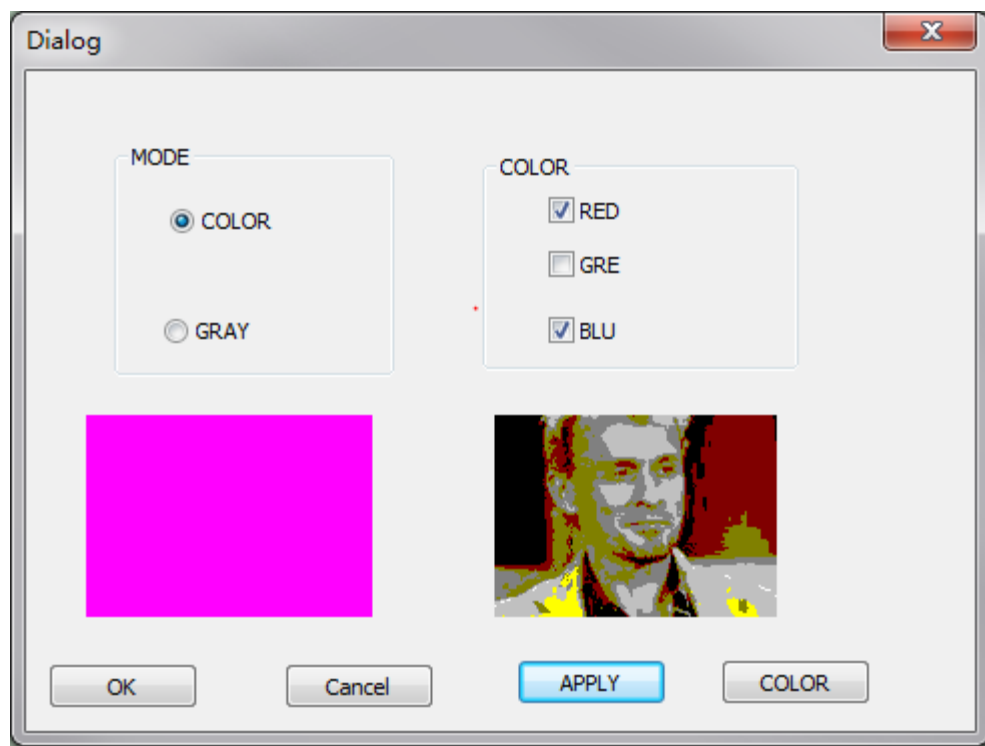
## 复选框

- ✓ 不同于单选按钮，在一组复选框中每次可以同时选择多项
- ✓ 除了选中和没选中两种状态，复选框还有第三种状态，此时选中标记显示为暗色，它表示该项不可以由用户选择
- ✓ 通过设置Tri-state属性得到这种三态复选框。
- ✓ 另外，我们可以通过设置Push-like属性使单选按钮或复选框具有普通按钮的外观。

## 按钮控件发送的消息：

- ✓ 按钮控件只能发送通知码为 **BN\_CLICKED**（单击按钮）和 **BN\_DOUBLECLICKED**（双击按钮）的 **WM\_COMMAND** 消息，我们经常需要编写按钮的 **BN\_CLICKED** 消息处理函数。
- ✓ **CButton** 类提供了一些成员函数实现对按钮控件对象的控制和管理，如利用成员函数 **GetCheck()** 或 **SetCheck()** 获取或设置单选按钮或复选框的当前状态。

例 编写一个对话框应用程序ColrButn，对话框中有两个用于选择颜色模式的单选按钮和三个用于选择具体颜色的复选框，只有在彩色模式下才能选择三种不同颜色的组合。当用户单击“应用”按钮，在对话框右侧绘制指定颜色的矩形。



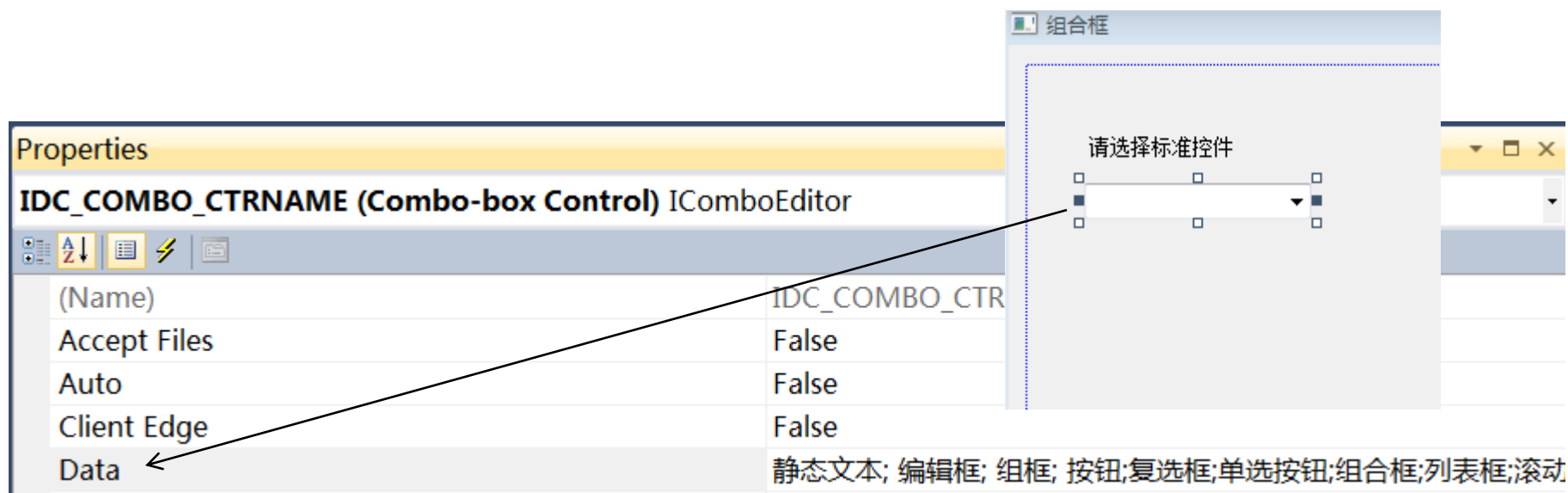
## 4) 组合框

- ✓ 组合框（**Combo box**）吸收了列表框和编辑框的优点
- ✓ 可以显示列表项供用户进行选择，
- ✓ 允许用户输入新的列表项。
- ✓ 实质上，组合框是多个控件的组合，包括编辑框、列表框和按钮。
- ✓ 通过设置组合框的**Type**属性可以使组合框有简单组合框（**Simple**）、下拉组合框（**Dropdown**）和下拉列表框（**Drop List**）三种形式

## 组合框控件的Data属性:

与编辑框或列表框相比，组合框有一个新的功能属性，可以通过组合框控件的Data属性添加初始的列表项，各个列表项用分号隔开。

也可以为组合框关联control类型的成员变量，并在在对话框类的初始化成员函数OnInitDialog()中编写代码添加列表项。



## 组合框发送的消息：

- ✓ 关闭组合框消息CBN\_CLOSEUP
- ✓ 打开列表框消息CBN\_DROPDOWN
- ✓ 双击列表项消息CBN\_DBLCLK
- ✓ 选择一个列表项并按下Enter键或单击下拉按钮隐藏列表框时发送消息CBN\_SELENDOK
- ✓ 当前选项被取消时（如重新选择其它控件或关闭对话框）发送消息CBN\_SELENDCANCEL
- ✓ 当前选项改变消息CBN\_SELCHANGE
- ✓ 当组合框的编辑框中的文本被修改且新的文本显示之后发送消息CBN\_EDITCHANGE
- ✓ 当编辑框中的文本被修改且新的文本显示之前发送消息CBN\_EDITUPDATE
- ✓ 组合框失去键盘输入焦点消息CBN\_KILLFOCUS
- ✓ 组合框获得键盘输入焦点消息CBN\_SETFOCUS。

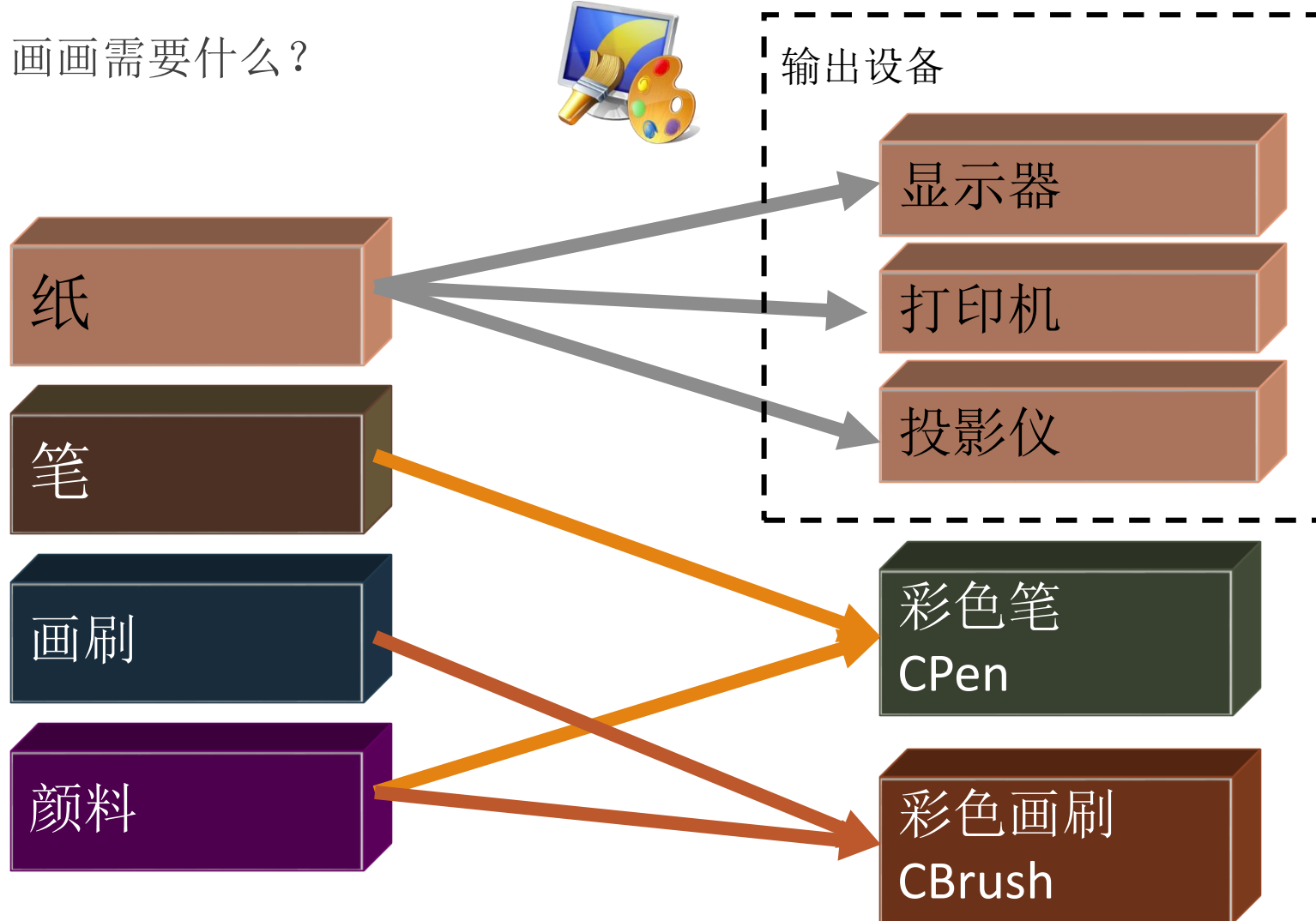


# 图形图像处理

---



画画需要什么？



## 图形设备接口（**Graphics Device Interface, GDI**）

- ✓ 不同图形设备的操作不一样（驱动不一样）,同类图形设备、不用品牌的操作也不一样;
- ✓ 程序设计者难道要针对每种情况编写程序?
- ✓ 如何使绘图程序适用于各种图形设备?

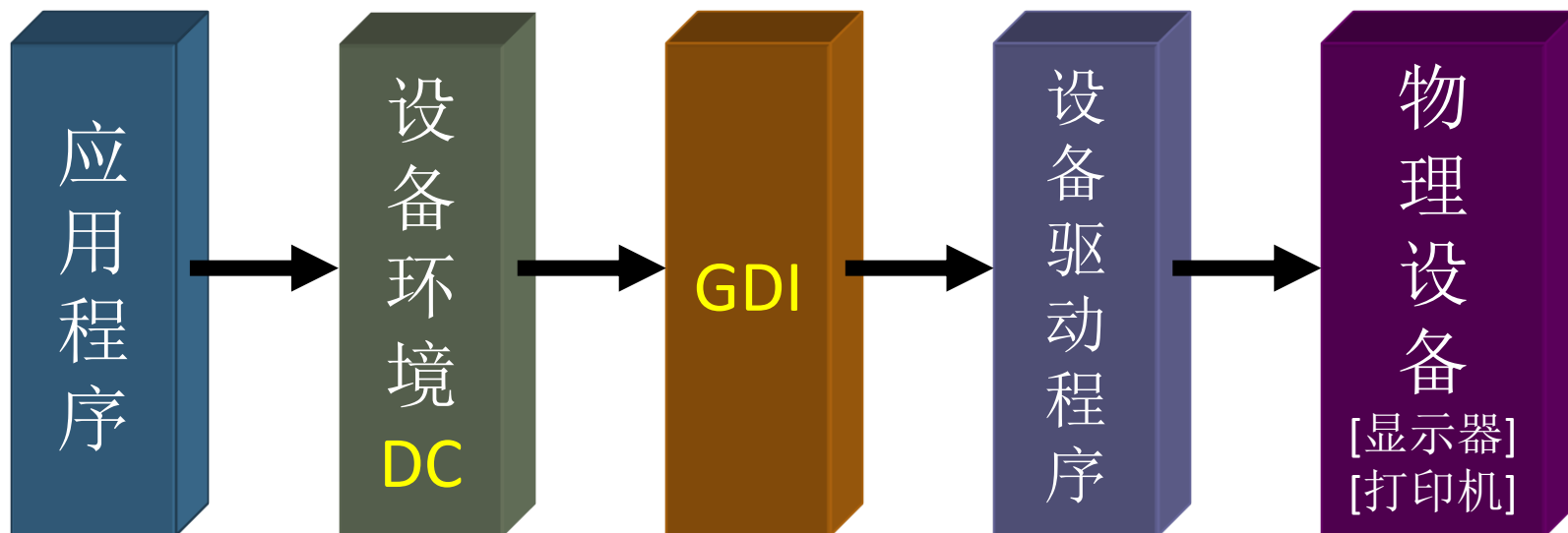
# GDI

# GDI概述

---

- ◆ GDI: Graphics Device Interface (图形设备接口), 它管理Windows应用程序图形的绘制, 在应用程序中, 通过调用GDI函数绘制不同尺寸、颜色、风格的几何图形、文本和位图。这些图形处理函数组成了图形设备接口GDI。
- ◆ MFC将GDI函数封装在一个名为CDC的设备环境类中, 可以通过调用CDC类的成员函数来完成绘图操作。

# 1、GDI



- GDI通过不同设备提供的驱动程序将绘图语句转换为对应的绘图指令
- 避免了绘图语句直接对硬件进行操作，从而实现设备无关性

## 2、设备环境（Device Context）

- 应用程序不直接面向显示器等物理设备，而是面向虚拟逻辑设备：**设备环境 (DC: Device Context)**。
- 设备环境也称**设备描述表**或**设备上下文**
  - Windows管理的一个数据结构
  - 保存了绘图操作中一些需要共同设置的信息，如当前的画笔、画刷、字体和位图等图形对象及其属性，以及颜色和背景等影响图形输出的绘图模式。
- 形象地说：设备环境提供**画布**和绘画**工具**，应用程序可以使用这些工具在画布上面绘制点、线、圆和文本。

# MFC编程时有哪些设备环境？

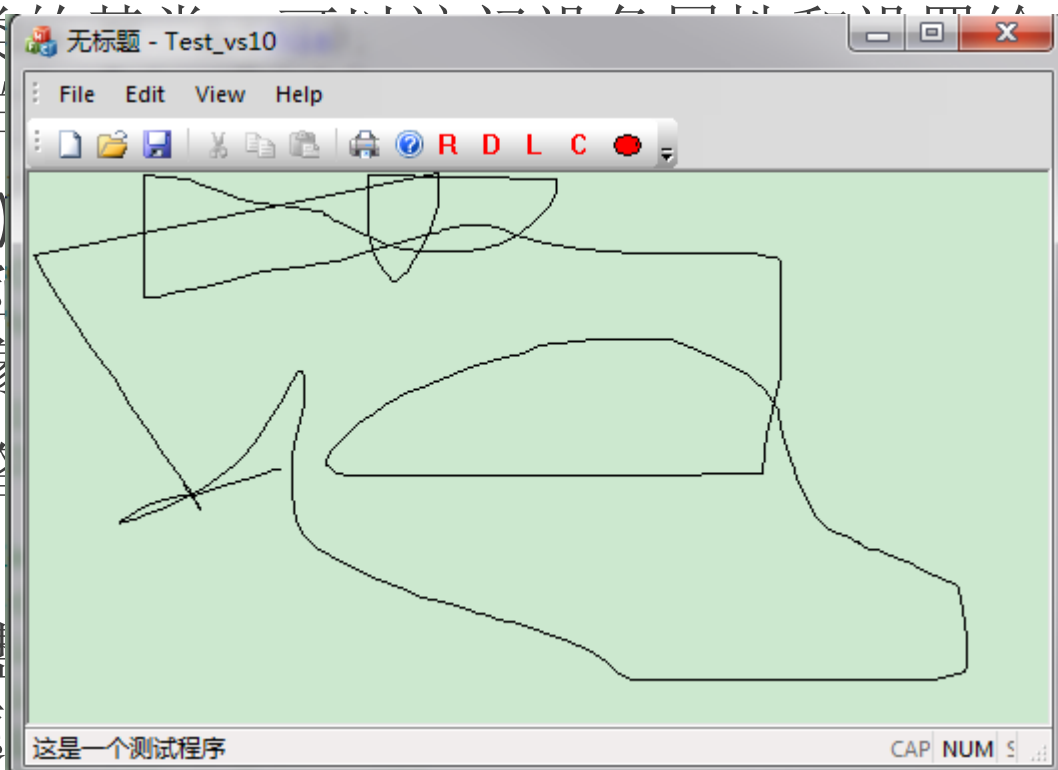


- **CDC:** 其它设备环境类的基类，可以访问设备属性和设置绘图属性。CDC类对GDI的所有绘图函数进行了封装。
- **CPaintDC:** 是OnPaint()函数使用的设备环境类。如果添加WM\_PAINT消息处理函数OnPaint(), 就需要使用CPaintDC类来定义一个设备环境对象。
- **CClientDC:** 客户区设备环境。在客户区实时绘图时需要定义的客户区设备环境。
- **CWindowDC:** 代表了整个程序窗口设备环境，可以在整个窗口区域绘图(包含标题栏, 工具条, 状态栏等)。

# MFC编程时有哪些设备环境?



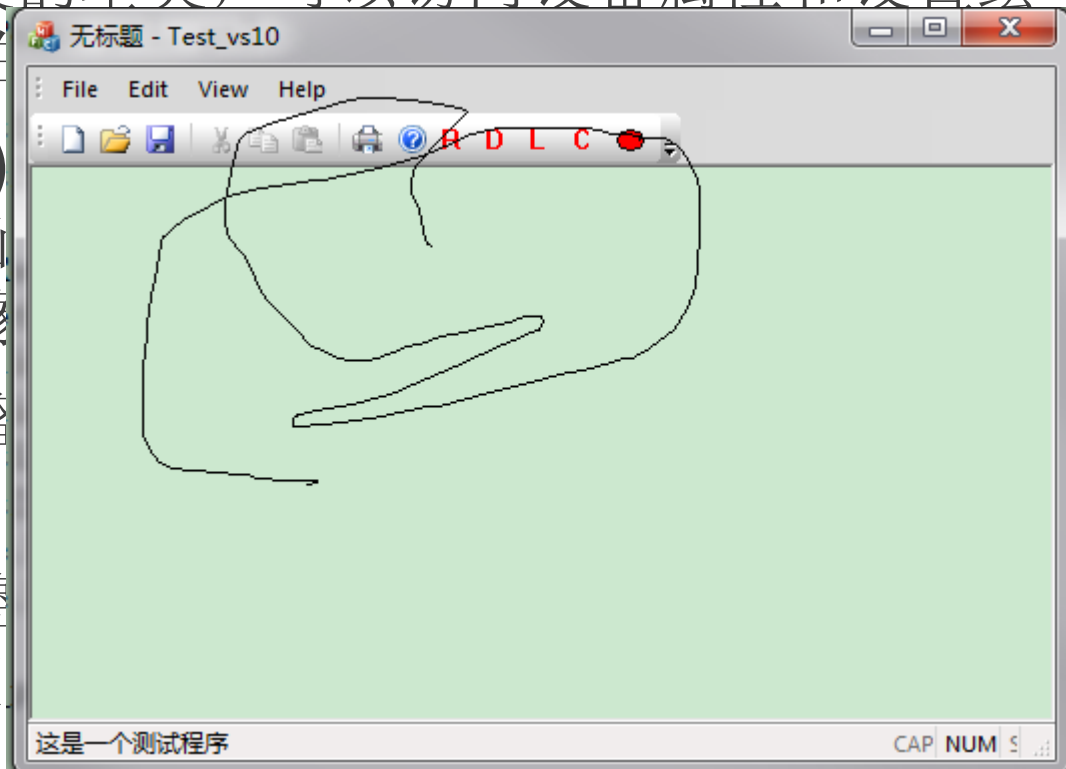
- **CDC:** 其它设备环境类的基础类。CDC类对GDI的绘图属性。CDC类对GDI的绘图属性。
- **CPaintDC:** 是OnPaint()消息处理函数中定义的一个设备环境对象。
- **CClientDC:** 客户区设备环境。客户区设备环境。
- **CWindowDC:** 代表了整个窗口区域绘图(包含标题栏和菜单)



# MFC编程时有哪些设备环境?



- **CDC:** 其它设备环境类的基类, 可以访问设备属性和设置绘图属性。CDC类对GDI的
- **CPaintDC:** 是OnPaint()消息处理函数中定义的一个设备环境对象
- **CClientDC:** 客户区设备环境, 用于客户区设备环境。
- **CWindowDC:** 代表了整个窗口区域绘图(包含标题



# 各种 D C 的区别

|           | 使用的<br>坐标系 | 是否可<br>被打印                                                                           | 一般在哪里使用             | 继承关系    |
|-----------|------------|--------------------------------------------------------------------------------------|---------------------|---------|
| CDC       | 逻辑坐标系      |    | OnDraw              | 设备环境基类  |
| CPaintDC  | 逻辑坐标系      |    | WM_PAINT<br>OnPaint | CDC的派生类 |
| CClientDC | 设备坐标系      |   | 其它函数                | CDC的派生类 |
| CWindowDC | 设备坐标系      |  | 所有函数                | CDC的派生类 |

# 怎样获取这些设备环境？

CDC主要用于OnDraw函数里；

CPaintDC主要用于WM\_PAINT消息的响应函数OnPaint里；

CClientDC用于其它函数里；

CWindowDC；

```
void CDfeView::OnLButtonDown(UINT nFlags, CPoint point)
{
    CWindowDC dc( this );
    dc.Rectangle(10,10,400,300);

    CView::OnLButtonDown(nFlags, point);
}
```

# SDK程序中怎样获取这些设备环境？

- ✓ **WM\_PAINT**消息处理函数中通过调用API函数**BeginPaint()**获取设备环境，在消息处理函数返回前调用API函数**EndPaint()**释放设备环境。
- ✓ 在其他函数中通过调用API函数**GetDC()**获取设备环境，调用API函数**ReleaseDC()**释放设备环境。

### 3、GDI坐标系

- ✓ Windows坐标系分为**逻辑坐标系**和**设备坐标系**两种，GDI支持这两种坐标系。一般而言，GDI的文本和图形输出函数使用逻辑坐标，而在客户区移动或按下鼠标的鼠标位置是采用设备坐标。
- ✓ **逻辑坐标系是面向DC的坐标系**，这种坐标不考虑具体的设备类型，在绘图时，Windows会根据当前设置的映射模式将逻辑坐标转换为设备坐标。
- ✓ **设备坐标系是面向物理设备的坐标系**，这种坐标以像素或设备所能表示的最小长度单位为单位，X轴方向向右，Y轴方向向下。设备坐标系的原点位置(0, 0)不限定在设备显示区域的左上角。

## 4、设备坐标系

设备坐标系分为屏幕坐标系、窗口坐标系和客户区坐标系三种相互独立的坐标系。

### ●坐标之间的相互转换

- MFC提供了两个函数 `CWnd::ScreenToClient()` 和 `CWnd::ClientToScreen()` 用于屏幕坐标与客户区坐标的相互转换。
- MFC提供了两个函数 `CDC::DPtoLP()` 和 `CDC::LPtoDP()` 用于设备坐标与逻辑坐标之间的相互转换。

# 映射模式

| 映射模式           | 逻辑单位     | 坐标系设定                |
|----------------|----------|----------------------|
| MM_TEXT        | 一个像素     | X轴正方向朝右，Y轴正方向朝下      |
| MM_LOMETRIC    | 0.1毫米    | X轴正方向朝右，Y轴正方向朝上      |
| MM_HIMETRIC    | 0.01毫米   | X轴正方向朝右，Y轴正方向朝上      |
| MM_LOENGLISH   | 0.01英寸   | X轴正方向朝右，Y轴正方向朝上      |
| MM_HIENGLISH   | 0.001英寸  | X轴正方向朝右，Y轴正方向朝上      |
| MM_TWIPS       | 1/1440英寸 | X轴正方向朝右，Y轴正方向朝上      |
| MM_ISOTROPIC   | 系统确定     | X、Y轴可任意调节，X、Y轴比例为1:1 |
| MM_ANISOTROPIC | 系统确定     | X、Y轴可任意调节，X、Y轴比例任意   |

# 坐标原点

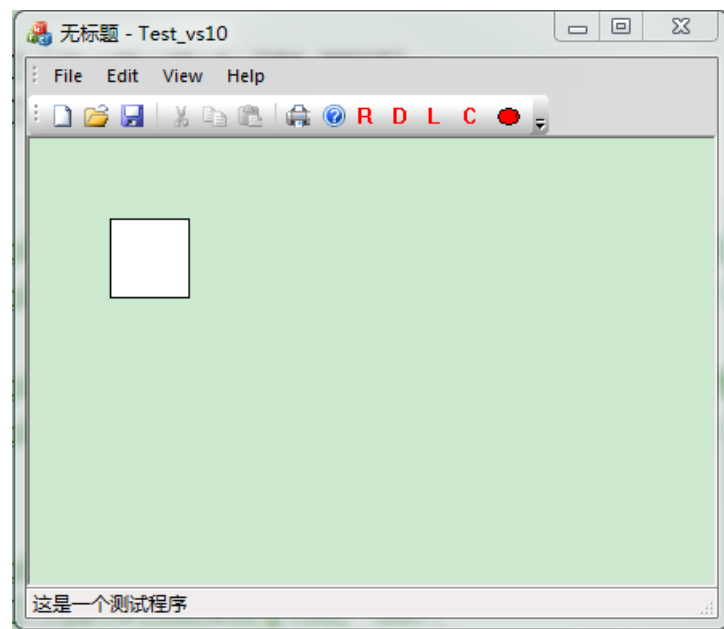
- ✓通过调用函数CDC::SetWindowOrg()设置设备环境的窗口原点的坐标，调用CDC::SetViewportOrg()重新设置设备的视口原点的坐标。
- ✓窗口原点是指逻辑窗口坐标系的原点在视口（设备）坐标系中的位置，视口原点是指设备实际输出区域的原点。
- ✓除了映射模式，窗口和视口也是决定一个点的逻辑坐标如何转换为设备坐标的一个因素。一个点的逻辑坐标按照如下式子转换为设备坐标：

$$\begin{aligned} \text{设备（视口）坐标} &= \text{逻辑坐标} - \text{窗口原点坐标} \\ &+ \text{视口原点坐标} \end{aligned}$$

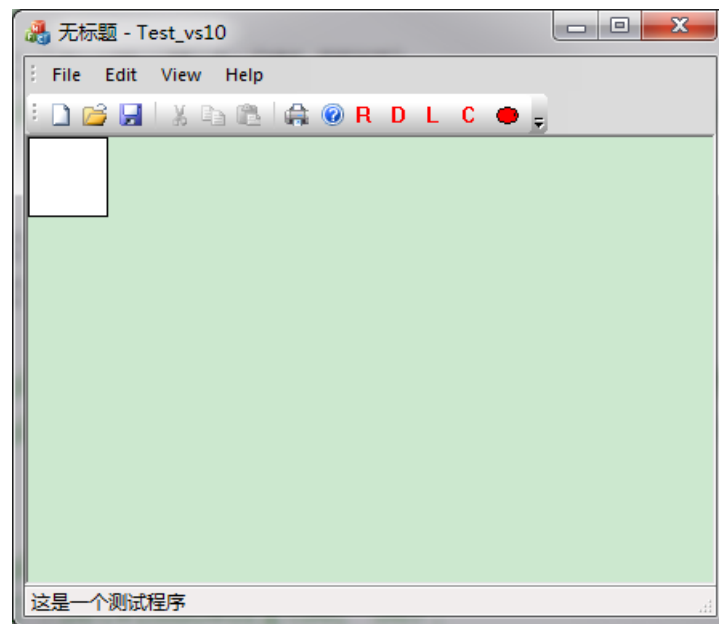
例 分别在OnDraw()函数中添加如下代码，设置不同的窗口原点和视口原点，结果有什么不同。

- (1)     pDC->SetMapMode(MM\_TEXT);  
          pDC->Rectangle(CRect(50, 50, 100, 100));
- (2)     pDC->SetMapMode(MM\_TEXT);  
          pDC->SetWindowOrg(50, 50); //屏幕左上角点坐标  
          pDC->Rectangle(CRect(50, 50, 100, 100));
- (3)     pDC->SetMapMode(MM\_TEXT);  
          pDC->SetViewportOrg(50,50); //输出的坐标原点  
          pDC->Rectangle(CRect(50, 50, 100, 100));
- (4)     pDC->SetMapMode(MM\_TEXT);  
          pDC->SetViewportOrg(50,50);  
          pDC->SetWindowOrg(50, 50);  
          pDC->Rectangle(CRect(50, 50, 100, 100));

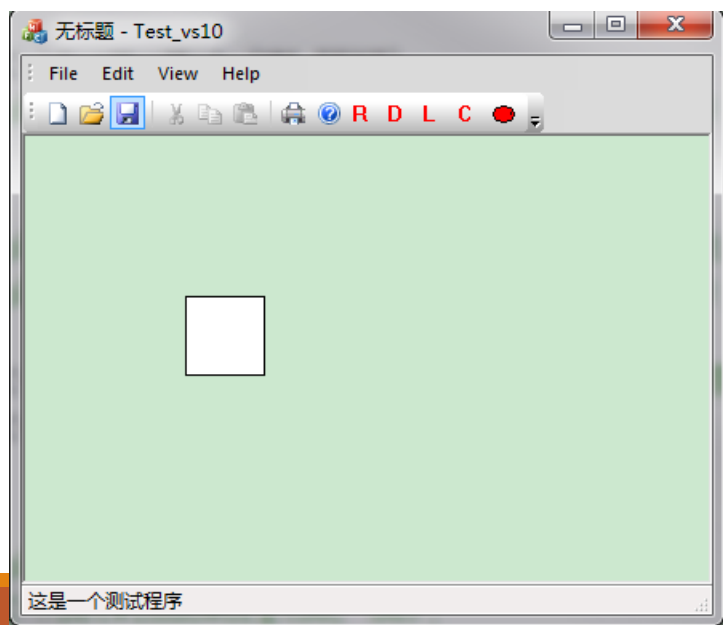
1



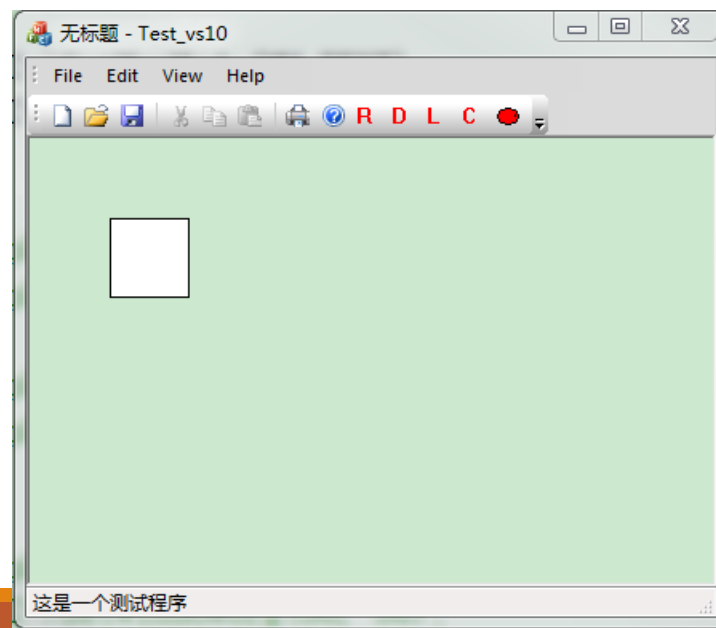
2



3



4



## 5、OnPaint 与 OnDraw的区别和联系

- ✓简单的说OnPaint只是负责窗口的重绘，而OnDraw是所有CDC的重绘（例如：打印机）
- ✓都是逻辑坐标绘图。（CClientDC为设备坐标绘图）
- ✓OnDraw是一个纯虚函数，定义为**virtual void OnDraw( CDC\* pDC ) = 0;** 而OnPaint是一个消息响应函数，它响应了WM\_PAINT消息，也是窗口重绘消息

获得DC后，就可以进行  
各种图形操作！



## 6、设置颜色

(1) Windows用**COLORREF**类型的数据存放颜色，它是一个32位整数。0x00000000~0x00FFFFFF共16777216种颜色。

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

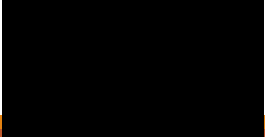
颜色由红、绿、蓝三种基本颜色组成，**COLORREF**类型数据的低位字节存放红色强度值，第2个字节存放绿色强度值，第3个字节存放蓝色强度值，高位字节为0

(2) Windows提供了**RGB**宏用于设置颜色：

**RGB(byRed, byGreen, byBlue)**

其中byRed、byGreen和byBlue分别表示红、绿、蓝分量值。

# 标准彩色的RGB值

| 颜色                                                                                     | RGB分量值        | 颜色                                                                                       | RGB分量值        |
|----------------------------------------------------------------------------------------|---------------|------------------------------------------------------------------------------------------|---------------|
|  浅红   | 255, 0, 0     |  深红   | 128, 0, 0     |
|  浅绿   | 0, 255, 0     |  深绿   | 0, 128, 0     |
|  浅蓝   | 0, 0, 255     |  深蓝   | 0, 0, 128     |
|  浅黄   | 255, 255, 0   |  深黄   | 128, 128, 0   |
|  浅青  | 0, 255, 255   |  深青  | 0, 128, 128   |
|  紫色 | 255, 0, 255   |  灰色 | 192, 192, 192 |
|  白色 | 255, 255, 255 |     | 0, 0, 0       |

# 定义颜色变量

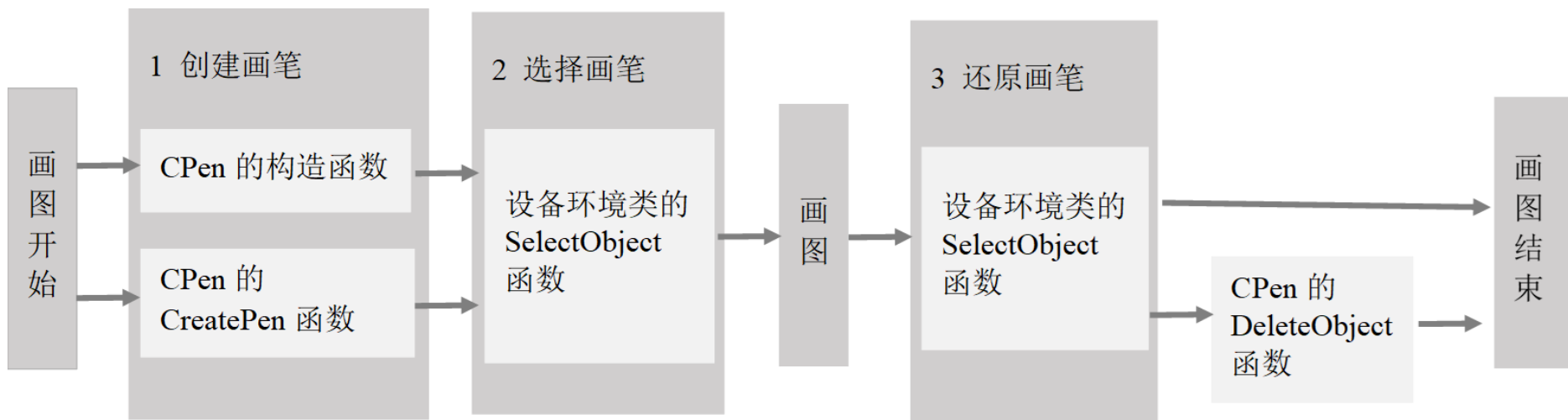
//定义绿色

COLORREF rgbBkClr=RGB(0,255,0);

//上句等同于

//COLORREF rgbBkClr=65280;

## 7、使用画笔 CPen





# 1)创建画笔

方法1：利用构造函数创建

```
CPen pen(PS_SOLID, 4, RGB(0, 0, 255));
```

方法2：利用CreatePen函数创建

```
CPen pen;
```

```
pen.CreatePen(PS_DASH, 1, RGB(255,0,0));
```

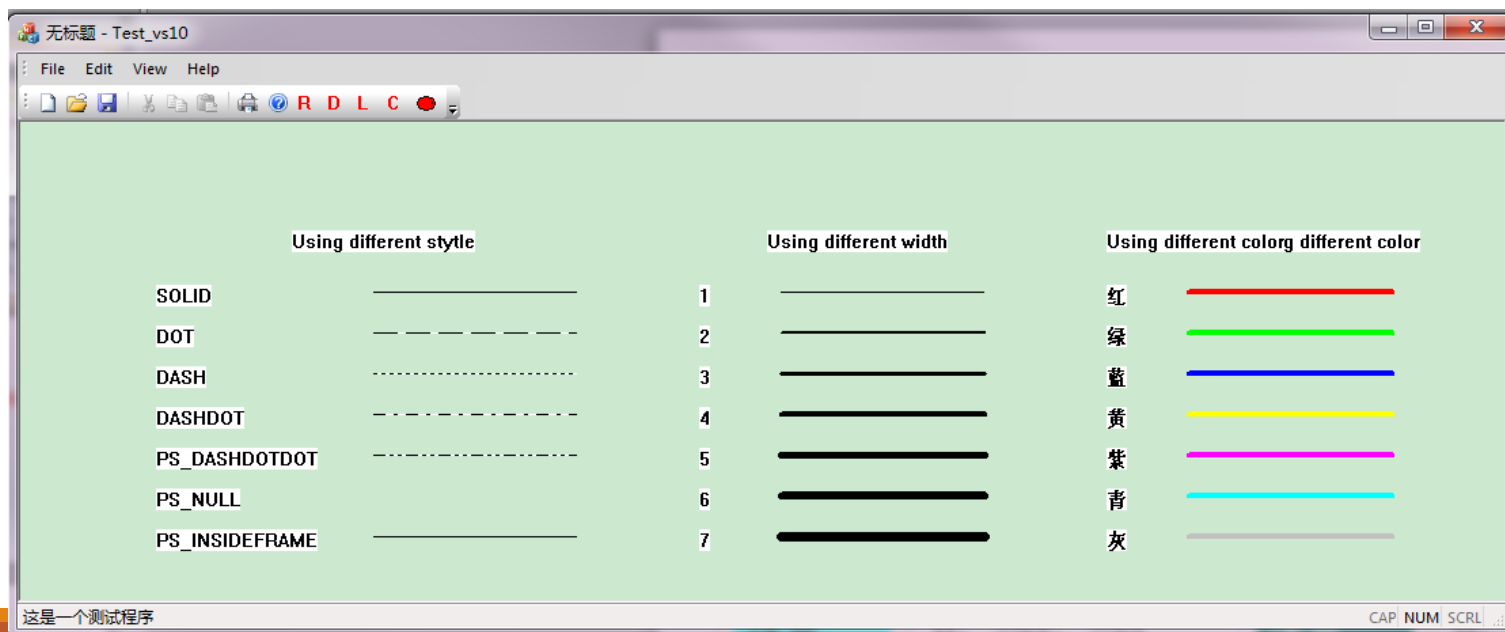
**注意：**以这种方式创建的画笔，如果以后不再使用，需要进行销毁。销毁方式是调用DeleteObject函数。

```
pen.DeleteObject();
```

画笔宽度只有在实线时才有效！其它样式的线只能是1个像素宽！

# 画笔样式

| 样 式        | 说 明 | 样 式            | 说 明  |
|------------|-----|----------------|------|
| PS_SOLID   | 实线  | PS_DASHDOTDOT  | 双点划线 |
| PS_DOT     | 点线  | PS_NULL        | 空的边框 |
| PS_DASH    | 虚线  | PS_INSIDEFRAME | 边框实线 |
| PS_DASHDOT | 点划线 |                |      |



## 2) 选择画笔

使用设备环境类的成员函数**SelectObject**来选择画笔作为当前使用的画笔。返回值为旧画笔的指针。

```
CPen penNew(PS_SOLID, 4, RGB(255,0,0));
```

```
CPen* pPenOld=pDC->SelectObject(&penNew);
```

## 3) 还原画笔

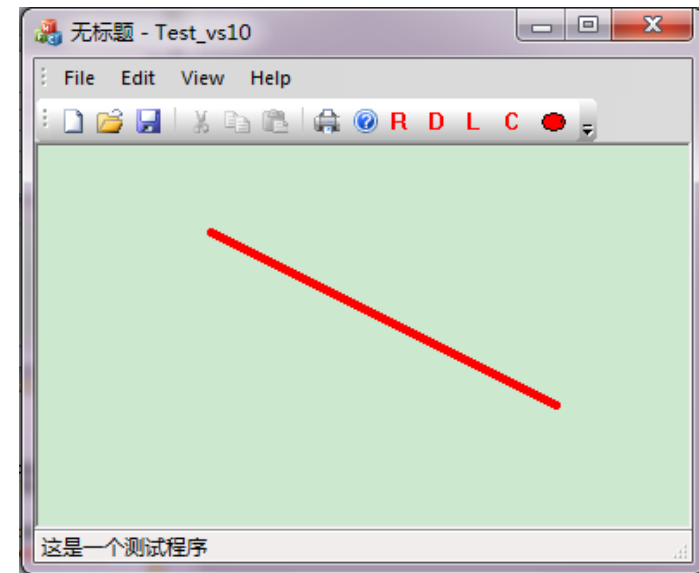
- 仍然使用SelectObject函数

```
pDC->SelectObject(pPenOld);
```



# 使用画笔的例子

```
//画图开始
//创建画笔
CPen penRed;
penRed.CreatePen(PS_SOLID, 5, RGB(255, 0, 0));
//选择新画笔，保存旧画笔
CPen *pOldPen = pDC->SelectObject(&penRed);
//画图：画一条直线
pDC->MoveTo(100, 50);
pDC->LineTo(300, 150);
//还原画笔
pDC->SelectObject(pOldPen);
//销毁画笔
penRed.DeleteObject();
//画图结束
```



## 备注

✓ 系统默认的画笔是**实线、1像素宽、黑色**。

✓ CDC函数

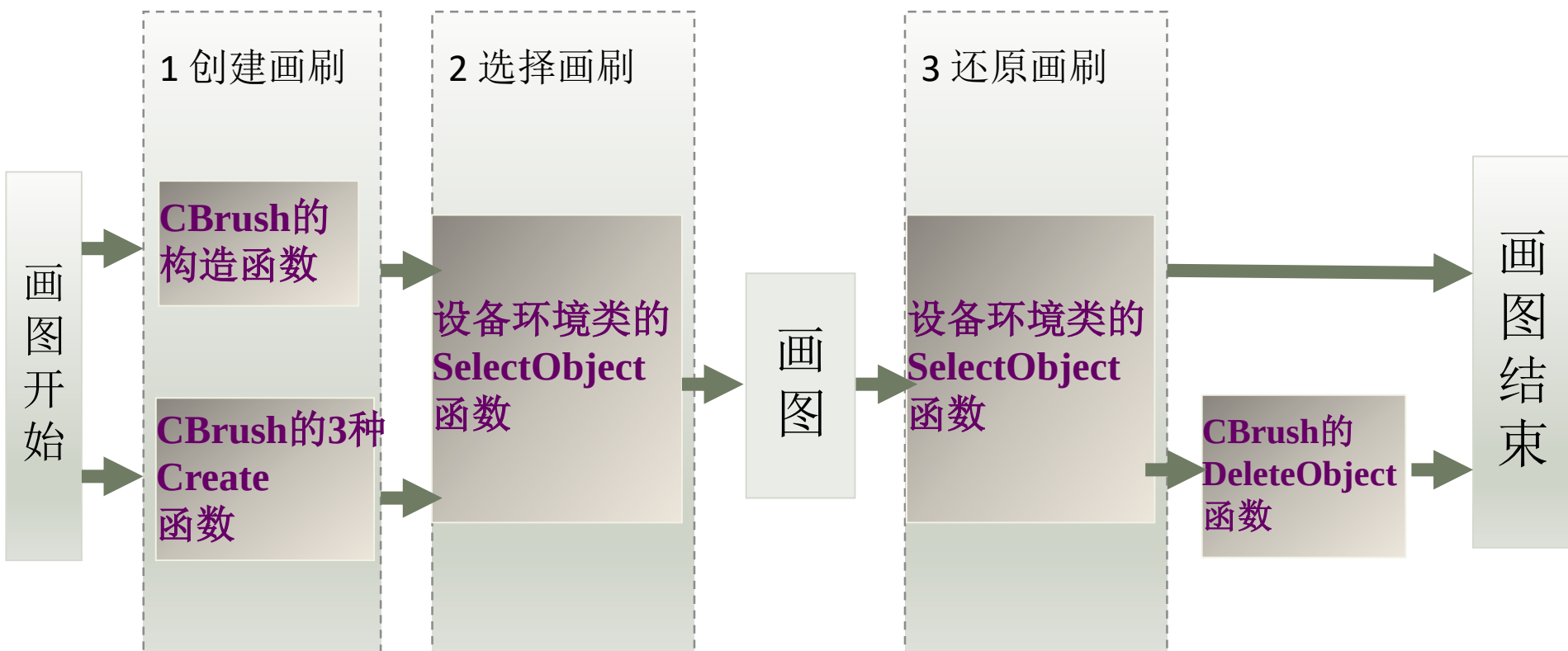
**MoveTo (int x, int y)**      //直线的起点

**LineTo (int x, int y)**      //直线的终点

以当前的画笔画一条直线。

## 8、使用画刷 CBrush

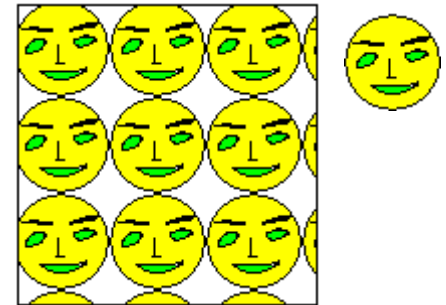
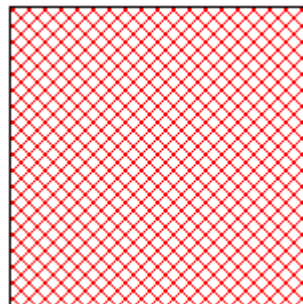
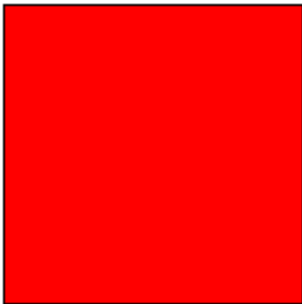
画刷能用来填充封闭的区域





## 画刷的类型

画刷有三种基本类型：纯色画刷、阴影画刷和图案画刷。



# 1) 创建画刷（构造函数方式和Create函数方式）

创建纯色画刷

```
CBrush brush1(RGB(255,0,0));  
//CBrush brush1;  
//brush1. CreateSolidBrush(RGB(255,0,0));
```

创建阴影画刷

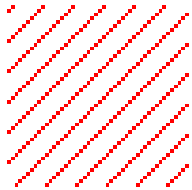
```
CBrush brush2(HS_DIAGCROSS, RGB(255,0,0));  
//CBrush brush2;  
//brush2. CreateHatchBrush(HS_DIAGCROSS, RGB(255, 0, 0));
```

创建图案画刷

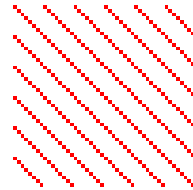
```
CBitmap bmp;  
bmp.LoadBitmap(IDB_BRUSH);  
CBrush brush3(&bmp);  
//CBrush brush3;  
//brush3. CreatePatternBrush(&bmp);
```

# 阴影画刷的样式

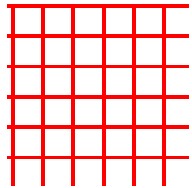
```
CBrush brush2(HS_DIAGCROSS, RGB(255,0,0));
```



HS\_BDIAGONAL



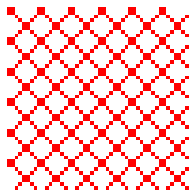
HS\_FDIAGONAL



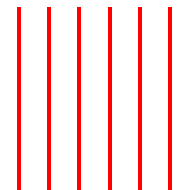
HS\_CROSS



HS\_HORIZONTAL



HS\_DIAGCROSS



HS\_VERTICAL

## 2) 选择画刷

- ◆ 使用设备环境类的成员函数 **SelectObject**
- ◆ 来选择画刷作为当前使用的画刷。返回值为旧画刷的指针。

**CBrush brushRed(RGB(255,0,0));**

**CBrush\* pBrushOld=pDC->SelectObject(&brushRed);**

## 3) 还原画刷

- 仍然使用 **SelectObject** 函数  
**pDC->SelectObject(pBrushOld);**

# 使用画刷的例子

//画图开始

//创建画刷

```
CBrush brush;
```

```
brush.CreateHatchBrush(HS_VERTICAL , RGB(255, 0, 0));
```

//保存旧画刷

```
CBrush *pOldBrush = pDC->SelectObject(&brush);
```

//画图：画矩形

```
pDC->Rectangle(50, 50, 200, 200);
```

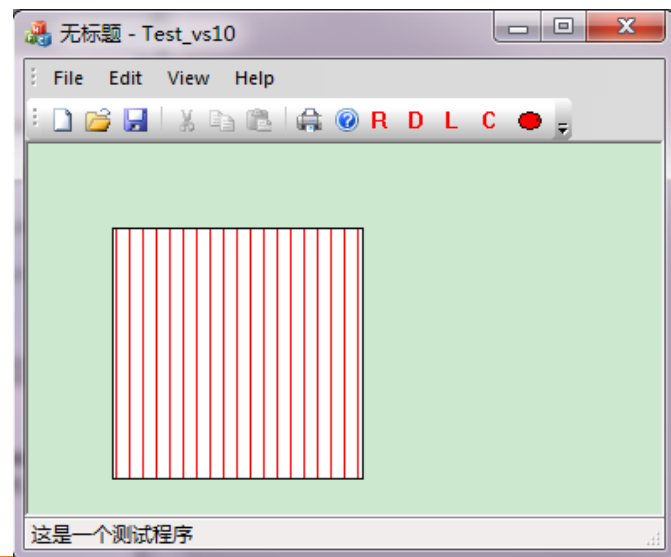
//还原画刷

```
pDC->SelectObject(pOldBrush);
```

//销毁画刷

```
brush.DeleteObject();
```

//画刷结束



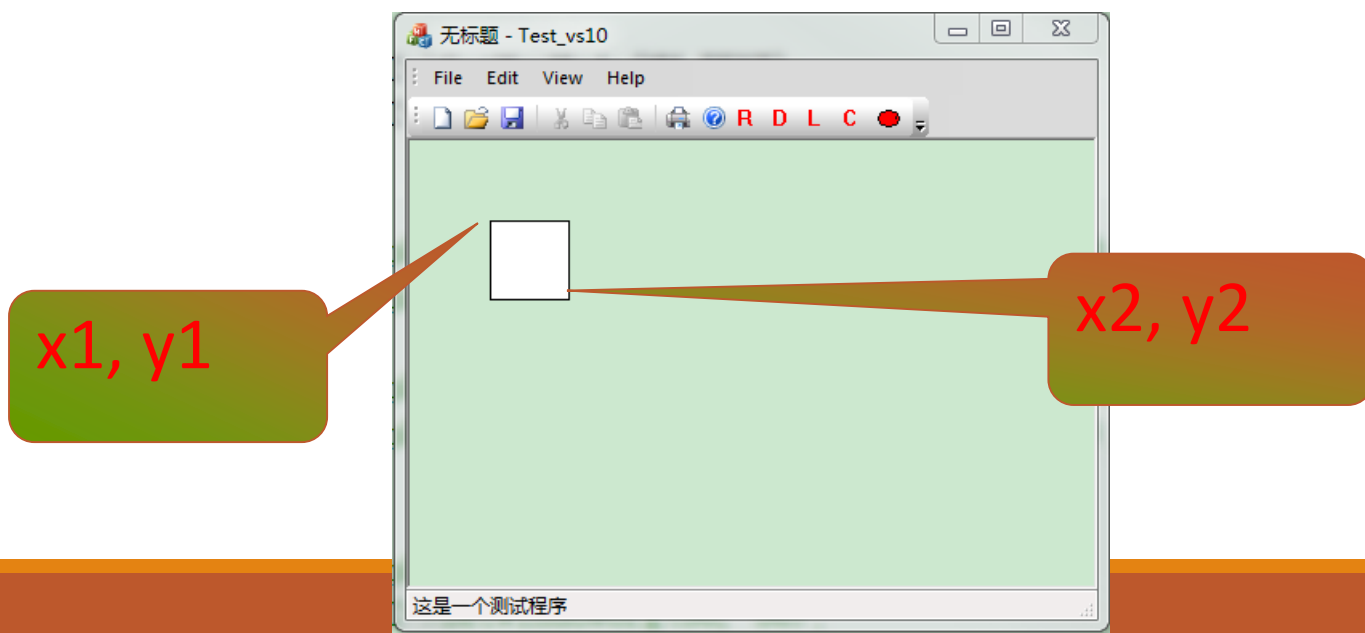
## 备注

✓ 系统默认画刷为纯色、白色的画刷。

CDC函数

**Rectangle (int x1, int y1, int x2, int y2)**

是以当前的画笔画矩形边框，内部以当前画刷填充



## 9、使用系统定义的画笔和画刷（堆对象）

Windows预定义了一些简单风格的GDI对象，用户使用这些GDI对象时，无需自己创建它们，可以直接将它们选入当前的设备环境，这些GDI对象称为**堆（Stock）对象**。堆对象包括**堆画笔、堆画刷和堆字体**等。

# 堆画笔、画刷的样式及说明

| 样 式                 | 说 明   | 样 式                 | 说 明   |
|---------------------|-------|---------------------|-------|
| <b>BLACK_PEN</b>    | 黑色画笔  | <b>WHITE_PEN</b>    | 白色画笔  |
| <b>NULL_PEN</b>     | 空画笔   | <b>BLACK_BRUSH</b>  | 黑色画刷  |
| <b>WHITE_BRUSH</b>  | 白色画刷  | <b>NULL_BRUSH</b>   | 空画刷   |
| <b>GRAY_BRUSH</b>   | 灰色画刷  | <b>DKGRAY_BRUSH</b> | 深灰色画刷 |
| <b>LTGRAY_BRUSH</b> | 浅灰色画刷 | <b>HOLLOW_BRUSH</b> | 虚画刷   |

# 使用堆对象

函数CDC::SelectStockObject()可以选择一个堆对象绘图工具，  
以下代码将堆画笔和堆画刷作为当前的绘图工具：

```
pPenOld=(CPen*) pDC->SelectStockObject(NULL_PEN);
```

```
// 使用堆画笔对象
```

```
pBrhOld=(CBrush*) pDC->SelectStockObject(LTGRAY_BRUSH);
```

```
// 使用堆画刷对象
```

## 10、绘制基本图形

- ✓ Windows中可以绘制的基本几何图形包括点、直线、曲线、矩形、椭圆、弧、扇形、弦形和多边形等。
- ✓ 绘图函数使用的坐标都是逻辑坐标。
- ✓ 各个DC类提供的绘图函数有：

**SetPixel**

**MoveTo**

**LineTo**

**Polyline**

**PolyBezier**

**Rectangle**

**RoundRect**

**Ellipse**

**Arc**

**ArcTo**

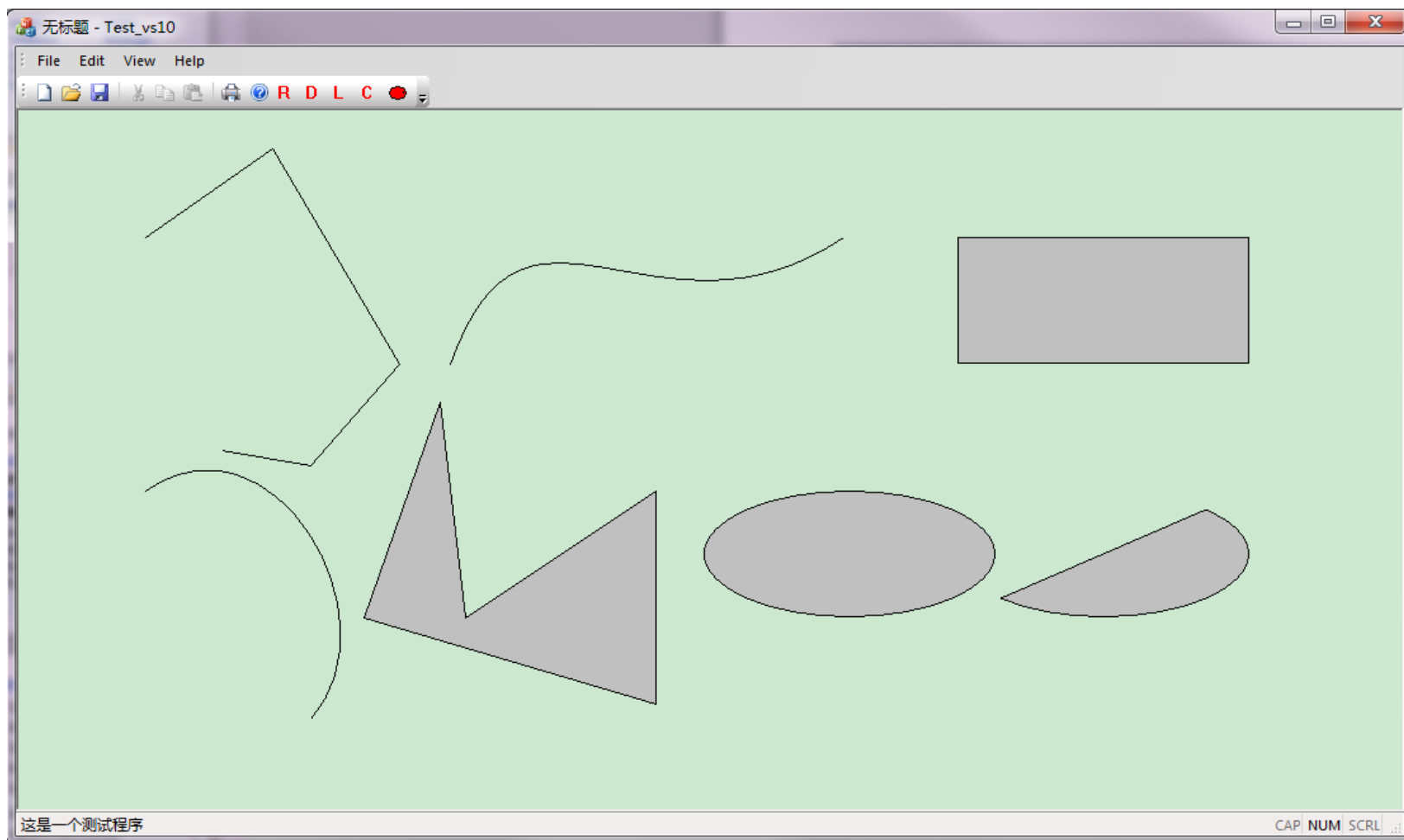
**Pie**

**Chord**

**Polygon**

**DrawIcon**

# 绘制基本图形



## 11、文本输出与字体

- ✓ 输出函数是**TextOut()**函数，该函数只能输出单行文本。要绘制多行文本可以调用**DrawText()**函数，另一个函数**ExtTextOut()**可以用一个矩形框对输出文本串进行裁剪。
- ✓ 在缺省情况下绘制文本时，字体颜色是**黑色**，背景颜色是**白色**，背景模式为**不透明**模式。可以通过调用CDC类成员函数重新设置字体颜色、背景颜色和文本对齐方式等文本属性。

# CDC类与文本输出相关的函数

|                |             |
|----------------|-------------|
| SetTextColor() | 设置显示文本的颜色   |
| GetTextColor   | 获得当前文本的颜色   |
| SetBkColor()   | 设置显示文本的背景颜色 |
| GetBkColor()   | 获得当前文本的背景颜色 |
| SetBkMode()    | 设置文本的背景模式   |
| GetBkMode()    | 获得当前文本的背景模式 |
| SetTextAlign() | 设置显示文本的对齐方式 |
| GetTextAlign() | 获得当前文本的对齐方式 |

# 字体

- ✓ 决定字体的三个要素是**字样**、**风格**和**大小**。字样是字母的样式和文本的视觉外观，字体的风格是字体的粗细和倾斜度。
- ✓ Windows支持**光栅字体**、**矢量字体**和**TrueType**三种字体。光栅字体即**点阵**字体，这种字体需要为每一种大小的字体创建独立的字体文件。矢量字体以一系列**线段**存储字符。TrueType字体是与设备无关的字体，字符以轮廓的形式存储，包括**线段和曲线**。

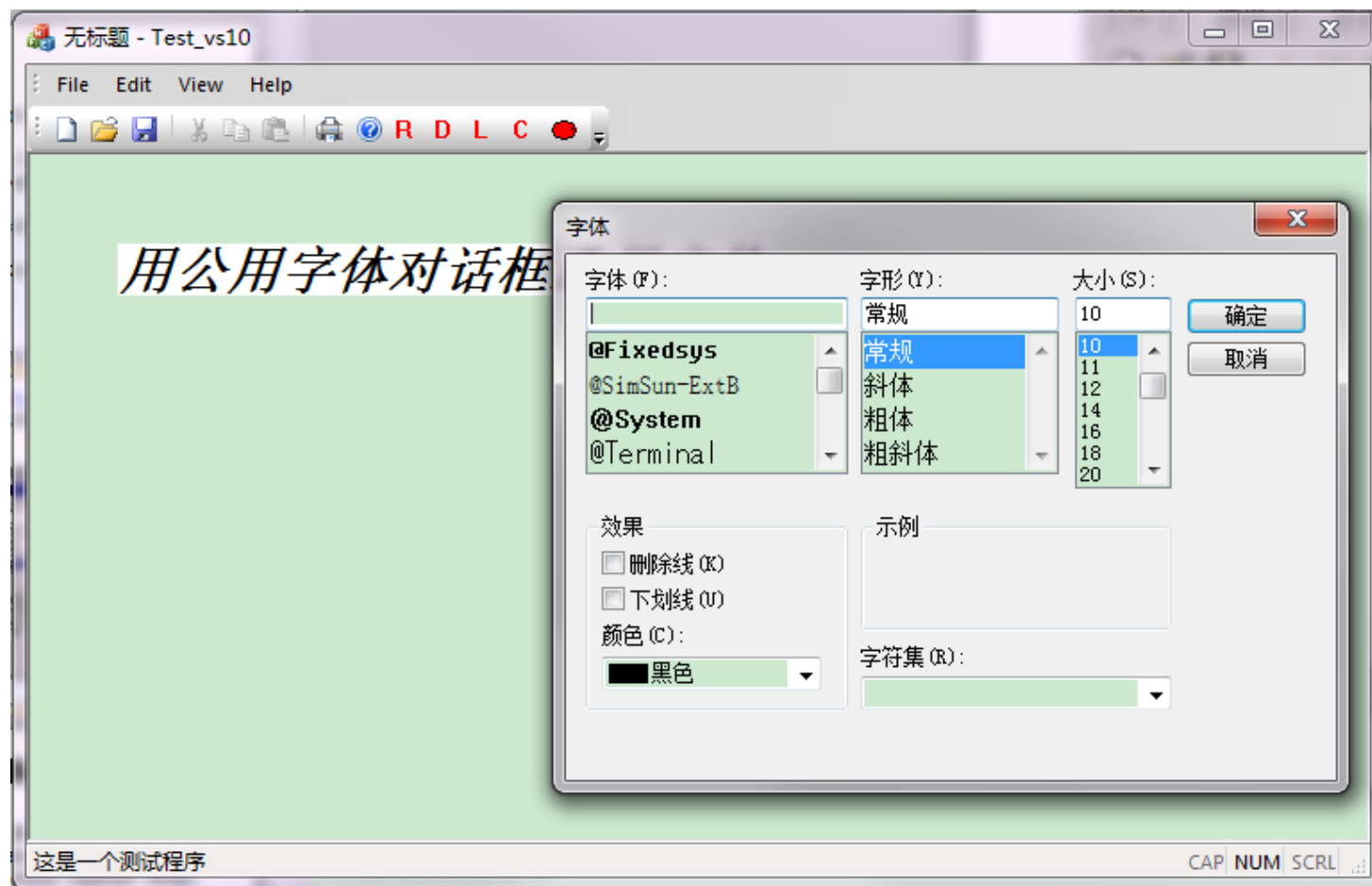
# 创建字体的方法:

1. 使用成员函数 `CFont::CreatPointFont()`
2. 使用成员函数 `CFont::CreateFontIndirect()`
3. 使用成员函数 `CFont::CreateFont()`

```
CClientDC dc(this);  
CFont fntZdy, *pfntOld;  
fntZdy.CreatePointFont(200, _T("Arial"), &dc);  
pfntOld=dc.SelectObject(&fntZdy); // 选入设备环境  
dc.TextOut(100, 100, "Hello! This is 20 Pt Arial Font.");  
dc.SelectObject(pfntOld);          // 恢复原来字体  
fntZdy.DeleteObject();             // 删除自定义字体
```

```
CFont font;  
LOGFONT LogFmt;  
memset(&LogFmt, 0, sizeof(LOGFONT));    // 清零结构LogFont  
LogFmt.lfHeight = 22;                    // 字体高度为22像素  
strcpy(LogFmt.lfFaceName, "Courier");    // 匹配字体为Courier  
font.CreateFontIndirect(&LogFmt);        // 创建字体  
CClientDC dc(this);  
CFont* def_font = dc.SelectObject(&font); // 选入设备环境  
dc.TextOut(100, 130, "Hello! This is 22-pixel-height Courier Font.");  
dc.SelectObject(def_font);  
font.DeleteObject();
```

使用公用字体对话框（[CFontDialog](#)）进行字体选择，根据用户选择的字体信息调用函数[CreateFontIndirect\(\)](#)创建字体



# 12、位图、图标和光标

## 1) 位图

两种不同形式的位图：

- 设备相关位图DDB(Device Dependent Bitmap)
- 设备无关位图DIB(Device Independent Bitmap)。

DIB位图通常以BMP文件形式保存在磁盘中，或者以资源形式存在于EXE或DLL执行文件中。

# MFC处理位图的方法:

- ✓ MFC只提供了处理DDB位图的类CBitmap，要显示DIB位图，可以先将一个DIB位图转换为DDB位图。
- ✓ 类CBitmap提供了一个成员函数，用于从程序的资源中装载位图，并可以将基于资源的DIB位图转换成GDI位图，该函数声明如下：

`BOOL LoadBitmap( LPCTSTR lpszResourceName );`

`BOOL LoadBitmap( UINT nIDResource );`

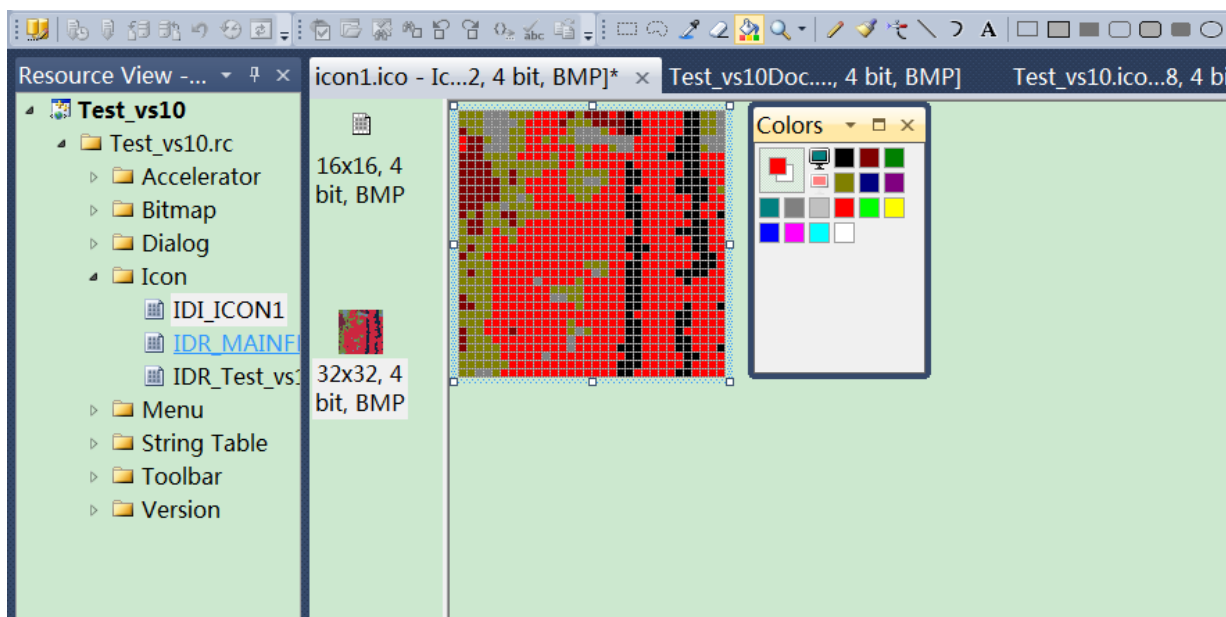
- 。 其中参数lpszResourceName或nIDResource分别为资源名称或资源标识，载入成功返回值为真，否则返回值为假。

# 显示位图的编程方法:

- ✓ 位图在显示之前必须先装入内存，把驻留在内存的位图数据送到显存时，位图就在显示器上显示
- ✓ 显示一个DDB位图步骤：
  - (1) 调用CDC::CreateCompatibleDC()创建一个兼容的内存设备环境；
  - (2) 调用CBitmap::LoadBitmap()装入位图资源或调用CBitmap::CreateCompatibleBitmap()创建一个与内存设备环境兼容的位图；
  - (3) 调用CDC::SelectObject()将位图选入设备环境；
  - (4) 调用CDC::BitBlt()或CDC::StretchBlt()将位图从内存设备环境中复制到指定设备如显示器。

## 2) 图标

- ✓ 在Windows中，每个文件都有一个图标。
- ✓ 图标实质上是特殊形式的位图。首先，图标大小尺寸只能有三种，一种是用标题栏和最小化时的 $16 \times 16$ 图标，另外两种是用于桌面、资源管理器的 $32 \times 32$ 和 $48 \times 48$ 图标。



# 图标的装载和显示

函数原型为：

```
HICON LoadIcon(LPCTSTR lpszResourceName) const;  
HICON LoadIcon(UINT nIDResource) const;
```

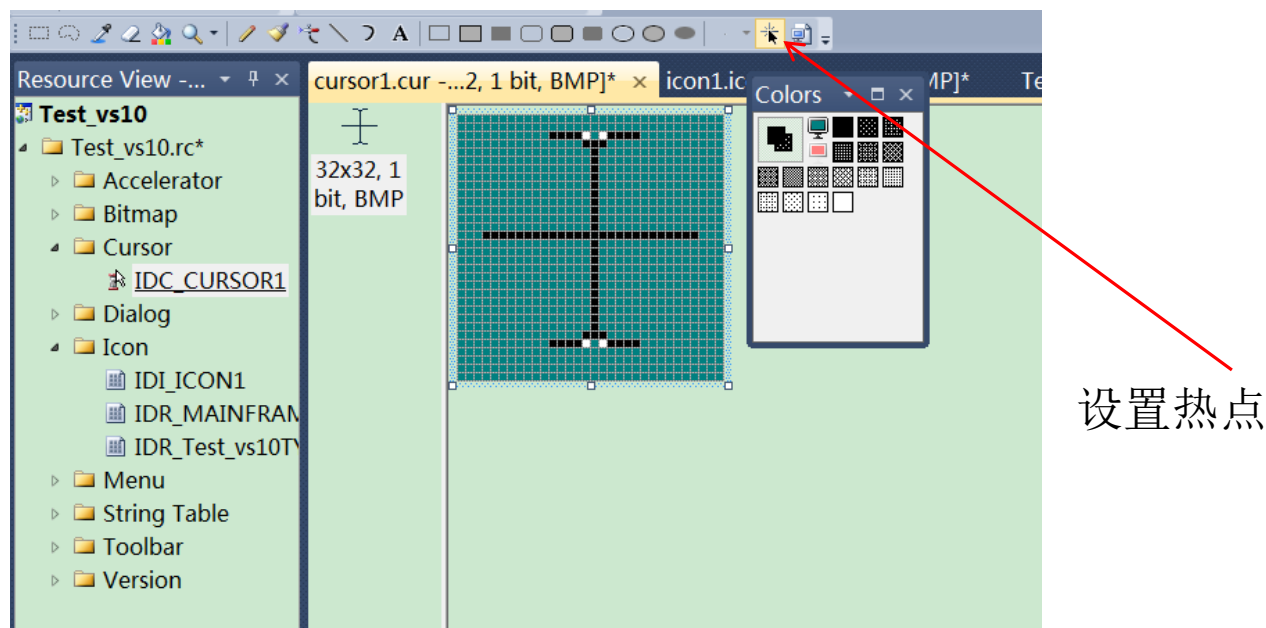
通过调用 `CWinApp::LoadStandardIcon()` 加载 Windows 系统提供的预定义图标。

显示

```
BOOL DrawIcon(int x, int y, HICON hIcon);  
BOOL DrawIcon(POINT point, HICON hIcon);
```

### 3) 光标

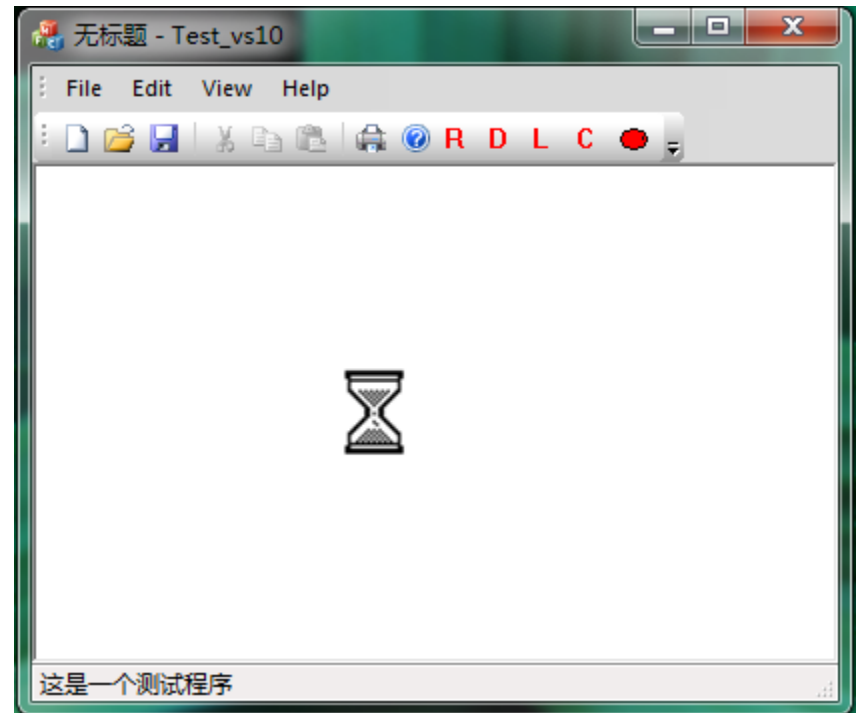
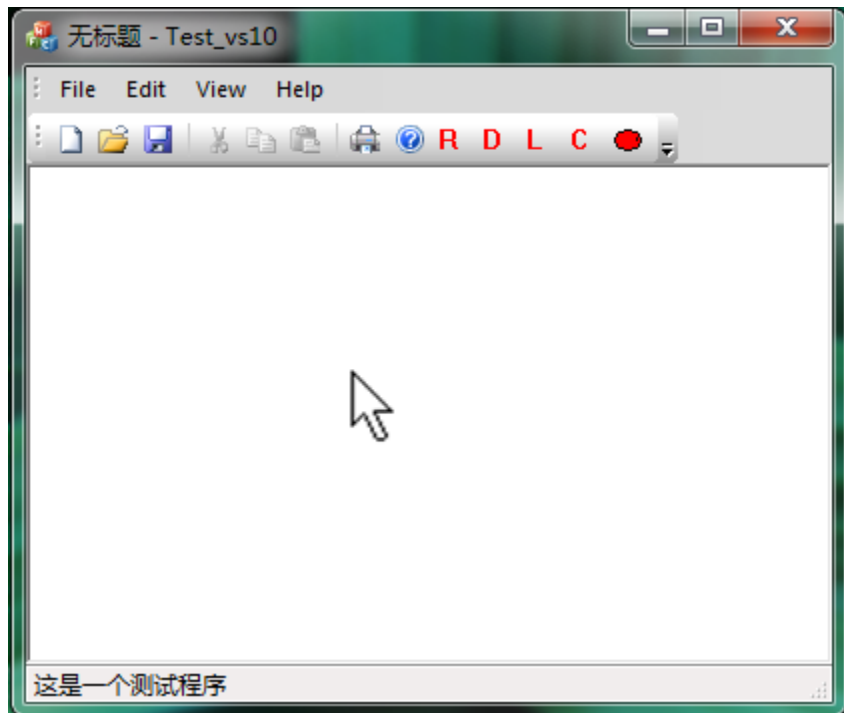
- ✓ 光标是一种特殊的、可移动的 $32 \times 32$ 点阵图形，它是用来作为鼠标指针的图形标志。
- ✓ 光标与图标最大的区别是光标有一个热点，用于确定光标当前的像素位置。



# 改变光标的形状

BeginWaitCursor()

EndWaitCursor()



## 例 编写基于对话框的程序，设置圆的半径和颜色，完成圆形的绘制。

新建基于对话框的应用程序，为对话框资源添加控件：

添加一个静态文本控件，其Caption为“请输入半径”。添加一个Edit Box编辑框控件，其ID标识为IDC\_EDIT\_RADIUS，该控件用于接收用户的输入的半径数据。

添加与控件关联的成员变量m\_nRadius，其数据类型为UINT。使用对话框数据验证DDV功能：m\_nRadius的最小值5和最大值250。

添加一个静态文本控件，其Caption为“请输入颜色”。添加一个Combo box编辑框控件，其ID标识为IDC\_EDIT\_COLOR，该控件用于接收用户的输入的颜色数据。

添加与控件关联的成员变量m\_nColor，其数据类型为Control。

- 1.在对话框类头文件定义三个int类型变量 m\_R, m\_G, m\_B。
- 2.在对话框类的OnInitDialog()中添加下面代码：

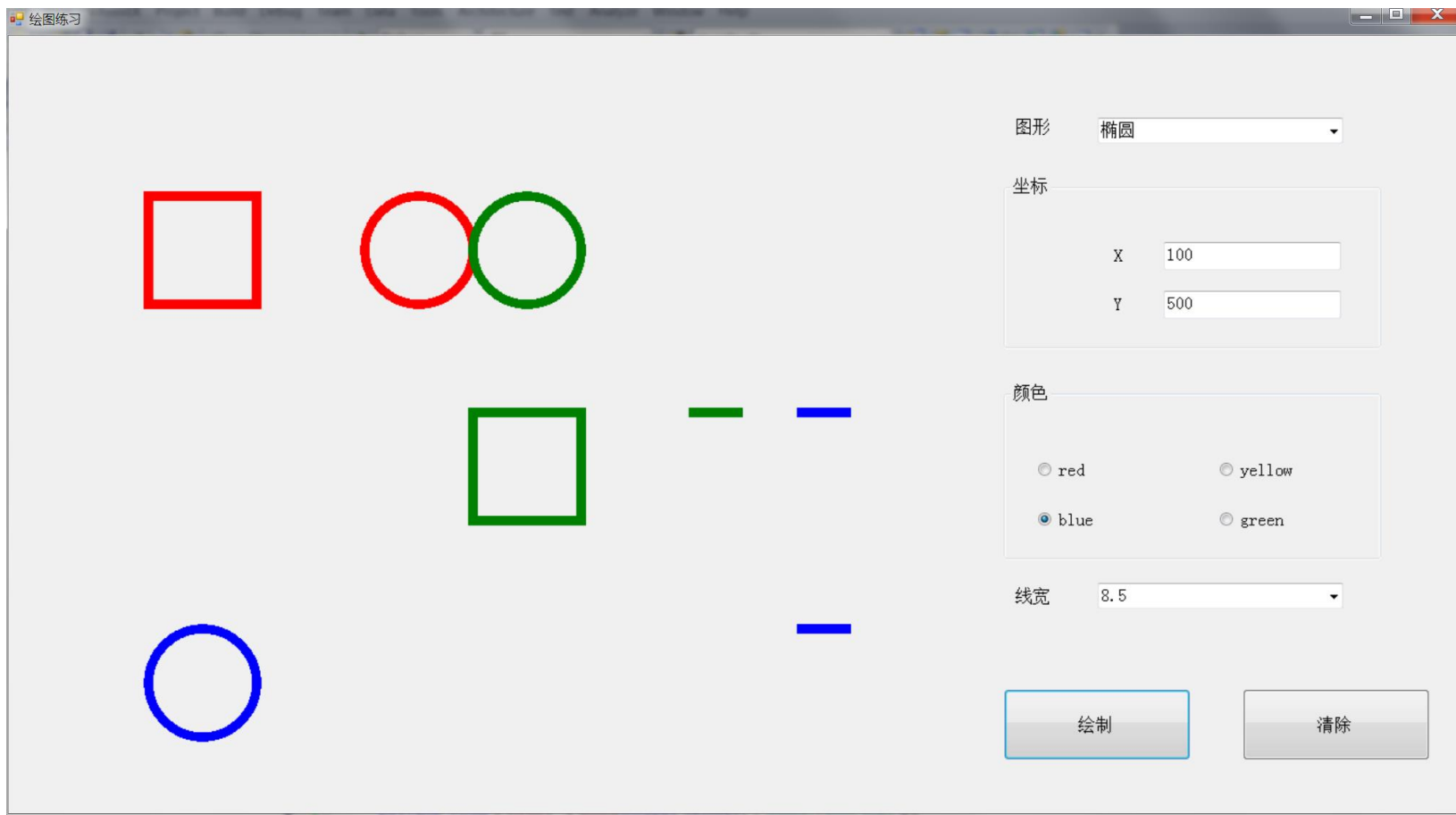
```
m_nRadius = 100; //初始化半径输入控件  
m_nColor.AddString(_T("red")); //初始化下拉框可选内容  
m_nColor.AddString(_T("green"));  
m_nColor.AddString(_T("blue"));  
m_nColor.SetCurSel(0); // 初始化下拉框选项  
m_R = 255;  
m_G = m_B = 0;
```

### 3. 在对话框类的OnPaint()中添加如下画圆的语句:

```
CPaintDC dc(this); // 用于绘制的设备上下文
CPen PenNew;
PenNew.CreatePen(PS_DASH, 1, RGB(m_R, m_G, m_B));
CPen* pPenOld;
pPenOld = dc.SelectObject(&PenNew);
dc.Ellipse(0,0,100,100);
dc.SelectObject(pPenOld); // 恢复设备环境DC中原来的画笔
PenNew.DeleteObject();    // 删除底层的GDI对象
```

# 任务一：

参照下图完成一个简单图形绘制程序的参数设置和图形绘制。



# 基本操作

1. 建立一个基于对话框的应用程序；
2. 在对话框中添加Static Text, Edit Control, Combo Box, Button, RadioButton等控件；
3. 为对话框类添加OnInitDialog函数，初始化参数；
4. 为对话框添加OnPaint，添加重绘代码
5. 为控件添加相关的处理函数，编写代码。

```
afx_msg void OnBnClickedDraw();
```

```
afx_msg void OnBnClickedClear();
```

```
afx_msg void OnRed();
```

```
afx_msg void OnGreen();
```

```
afx_msg void OnBlue();
```

```
afx_msg void OnYellow();
```

```
afx_msg void OnSelchangeLinewidth();
```

```
afx_msg void OnSelchangeType();
```

# 参考代码

```
#include <vector>
#include <atltypes.h>
using namespace std;
struct ColorStruct {
    int m_r;    int m_g;    int m_b;
};
class Shapeself {
public:
    int linewidth;
    ColorStruct linecolor;
    int mode;
    vector<CPoint> pointlist;
    Shapeself(vector<CPoint> pt, int lw, ColorStruct cl, int md) {
        pointlist = pt;
        linewidth = lw;
        linecolor = cl;
        mode = md;
    }
};
```

为对话框类添加数据成员：

```
int m_r, m_g, m_b;    //颜色  
vector<Shapeseif> shapelist;  //记录绘图信息  
int m_index_type;    //图形形状编号  
int m_ilinewidth;    //线宽
```

为对话框类添加控件关联变量

```
CComboBox m_type;  
CComboBox m_linewidth;  
CButton m_red;  
int m_x;  
int m_y;    //坐标
```

```
BOOL CDrawDlg::OnInitDialog()
{
    CDialogEx::OnInitDialog();
    // TODO: 在此添加额外的初始化
    m_type.SetCurSel(m_index_type);
    m_index_type = 0;
    m_linewide.SetCurSel(3);
    m_ilinewide = m_linewide.GetCurSel();
    m_r = 255;
    m_g = 0;
    m_b = 0;
    m_x = 100;
    m_y = 50;
    UpdateData(FALSE);
    m_red.SetCheck(1);

    return TRUE; // return TRUE unless you set the focus to a control
    // 异常: OCX 属性页应返回 FALSE
}
```

```
void CDrawDlg::OnBnClickedDraw()
{
    // TODO: 在此添加控件通知处理程序代码
    UpdateData(TRUE);
    CClientDC dc(this);
    CPen PenNew;
    CPen* pPenOld;
    PenNew.CreatePen(PS_DASH, m_ilinewide, RGB(m_r, m_g, m_b));
    pPenOld = dc.SelectObject(&PenNew);
    ColorStruct col={ m_r,m_g,m_b };
    vector<CPoint> pointlist;
    CPoint pt1, pt2;
    switch (m_index_type)
    {
    case 1:
        pt1 = CPoint(m_x, m_y);
        pt2 = CPoint(m_x + 50, m_y + 50);
        dc.Ellipse(pt1.x,pt1.y,pt2.x,pt2.y);
        break;
```

case 2:

```
pt1 = CPoint(m_x, m_y);  
pt2 = CPoint(m_x + 50, m_y + 50);  
dc.Rectangle(pt1.x, pt1.y, pt2.x, pt2.y);  
break;
```

case 0:

```
pt1 = CPoint(m_x, m_y);  
pt2 = CPoint(m_x + 50, m_y + 50);  
dc.MoveTo(pt1);  
dc.LineTo(pt2);  
break;
```

```
}
```

```
pointlist.push_back(pt1);
```

```
pointlist.push_back(pt2);
```

```
shapelist.push_back(Shapeself(pointlist, m_ilinewide, col, m_index_type));
```

```
dc.SelectObject(pPenOld); // 恢复设备环境DC中原来的画笔
```

```
PenNew.DeleteObject(); // 删除底层的GDI对象
```

```
}
```

```
void CDrawDlg::OnBnClickedClear()
{
    // TODO: 在此添加控件通知处理程序代码
    shapelist.clear();
}
```

```
void CDrawDlg::OnSelchangeLinewide()
{
    // TODO: 在此添加控件通知处理程序代码
    m_ilinewide = m_linewide.GetCurSel();
}
```

```
void CDrawDlg::OnSelchangeType()
{
    // TODO: 在此添加控件通知处理程序代码
    m_index_type = m_type.GetCurSel();
}
```

```
void CDrawDlg::OnRed(){
    // TODO: 在此添加命令处理程序代码
    m_r = 255;    m_g = 0;    m_b = 0;
}
void CDrawDlg::OnGreen()
{
    // TODO: 在此添加命令处理程序代码
    m_r = 0;    m_g = 255;    m_b = 0;
}
void CDrawDlg::OnBlue()
{
    // TODO: 在此添加命令处理程序代码
    m_r = 0;    m_g = 0;    m_b = 255;
}
void CDrawDlg::OnYellow()
{
    // TODO: 在此添加命令处理程序代码
    m_r = 255;    m_g = 255;    m_b = 0;
}
```

```
void CDrawDlg::OnPaint() {  
    //.....  
    if (shapelist.size()>0) {  
        for (int i = 0; i < shapelist.size(); i++) {  
            CPen PenNew;  
            PenNew.CreatePen(PS_DASH, shapelist[i].linewidth,  
                RGB(shapelist[i].linecolor.m_r, shapelist[i].linecolor.m_g,  
                    shapelist[i].linecolor.m_b));  
            CPen* pPenOld;  
            pPenOld = dc.SelectObject(&PenNew);  
            switch (shapelist[i].mode)  
            {  
            case 0:  
                dc.MoveTo(shapelist[i].pointlist[0].x,  
                    shapelist[i].pointlist[0].y);  
                dc.LineTo(shapelist[i].pointlist[1].x, shapelist[i].pointlist[1].y);  
                break;
```

```
case 1:
    dc.Ellipse(shapelist[i].pointlist[0].x, shapelist[i].pointlist[0].y,
shapelist[i].pointlist[1].x,shapelist[i].pointlist[1].y);
    break;
case 2:
    dc.Rectangle(shapelist[i].pointlist[0].x,
shapelist[i].pointlist[0].y, shapelist[i].pointlist[1].x,
shapelist[i].pointlist[1].y);
    break;
}
dc.SelectObject(pPenOld); // 恢复设备环境DC中原来的画笔
PenNew.DeleteObject(); // 删除底层的GDI对象
}
}
}
```



# 鼠标响应与定时器

---



# 鼠标消息处理

- Windows是基于**事件驱动**、**消息传递**的操作系统。用户所有的输入都是以事件或消息的形式传递给应用程序的，鼠标也不例外。
- **鼠标驱动程序**将鼠标硬件信号转换成Windows可以识别的信息，Windows根据这些信息构造鼠标消息，并将它们发送到应用程序的消息队列中。
- 只要鼠标跨越窗口或者在某个窗口中按下鼠标键，窗口就会接收到鼠标消息，而不管该窗口是否活动或带有输入焦点。

# 鼠标消息

- 当用户移动鼠标或者释放鼠标时，将产生鼠标消息。鼠标消息可分成两类：**客户区鼠标消息**和**非客户区鼠标消息**。
  - **非客户区**包括窗口的边界、标题栏、菜单、滚动条、最大化/最小化按钮。**非客户区鼠标消息**指的是鼠标在这些区域的操作而产生的消息，这种消息一般由系统处理。
  - 应用程序主要处理鼠标在**客户区**的操作而产生的消息。
- 通过消息结构中的消息参数**wParam**来区分客户区鼠标消息和非客户区鼠标消息。



# 主要鼠标消息：

WM\_MOUSEMOVE： 移动

WM\_LBUTTONDOWN： 按下左键

WM\_LBUTTONUP： 释放左键

WM\_RBUTTONDOWN： 按下右键

WM\_RBUTTONUP： 释放右键

WM\_LBUTTONDBLCLK： 双击左键

WM\_RBUTTONDBLCLK： 双击右键

鼠标操作： 单击、双击、释放和移动

# 鼠标消息处理函数参数

- 利用MFC ClassWizard类向导可以生成的鼠标消息处理函数，如：

**void OnLButtonDbLClk(UINT nFlags, CPoint point);**

**void OnLButtonDown(UINT nFlags, CPoint point);**

**void OnLButtonUp(UINT nFlags, CPoint point);**

**void OnMouseMove(UINT nFlags, CPoint point);**

- 一般都有两个参数：

- **point:** 指出光标当前所在位置坐标，以客户区的左上角为原点，X轴向右增加，Y轴向下增加。



# 鼠标消息处理函数参数

✓ **nFlags**: 返回虚拟码，代表产生鼠标事件时鼠标按键和几个特殊键盘键(Alt、Shift和Ctrl)的状态。键是否按下由 **nFlags** 参数的特定位表示。该参数可能包含的值有：

**MK\_CONTROL** Set if the **CTRL** key is down.

**MK\_LBUTTON** Set if **the left mouse button** is down.

**MK\_MBUTTON** Set if **the middle mouse button** is down.

**MK\_RBUTTON** Set if **the right mouse button** is down.

**MK\_SHIFT** Set if the **SHIFT** key is down.

◆ 例如检测Shift键和Ctrl键的状态，如下：

```
void OnLButtonDown(UINT nFlags,CPoint point)
```

```
{
```

```
    //判断是否Ctrl键和Shift键同时按下
```

```
    if((nFlags&MK_CONTROL)&&(nFlags&MK_SHIFT))
```

```
    {
```

```
        MessageBox(_T("您按下了鼠标右键，并同时按下了control  
和shift键！"));
```

```
    }
```

```
}
```

例：对前面建的对话框应用程序，增加  
WM\_LBUTTONDOWN  
消息，并弹窗显示鼠标左键的点击次数。

# 鼠标捕捉

---

如果你的鼠标移出了某个窗口的客户区范围，那么此窗口将不再继续收到客户区鼠标消息，会带来意想不到的后果。

如果你用SetCapture进行了鼠标捕捉，那么，即使你的鼠标移出了窗口客户区，仍然可以收到客户区鼠标消息，直到调用ReleaseCapture释放鼠标捕捉或在某个窗口上进行点击为止。

✓实现鼠标的捕捉的SetCapture函数的原型如下：

```
HWND SetCapture(HWND hWnd);
```

hWnd为要捕捉鼠标消息的窗口句柄。

✓如果在程序中执行如下语句：

```
SetCapture(hWnd);
```

这将导致Windows向窗口句柄为hWnd的窗口发送所有的鼠标消息。

□注意：一旦某个窗口捕获了鼠标，其他窗口将无法得到鼠标消息，因此，当窗口不再需要捕获鼠标消息时，应及时使用ReleaseCapture函数将鼠标释放。

## 例：鼠标消息处理与绘图

---

在任务一的基础上编写绘图程序，程序运行后，当用户在对话框窗口按下鼠标左键并移动时，根据鼠标移动的轨迹绘制出指定的线段。

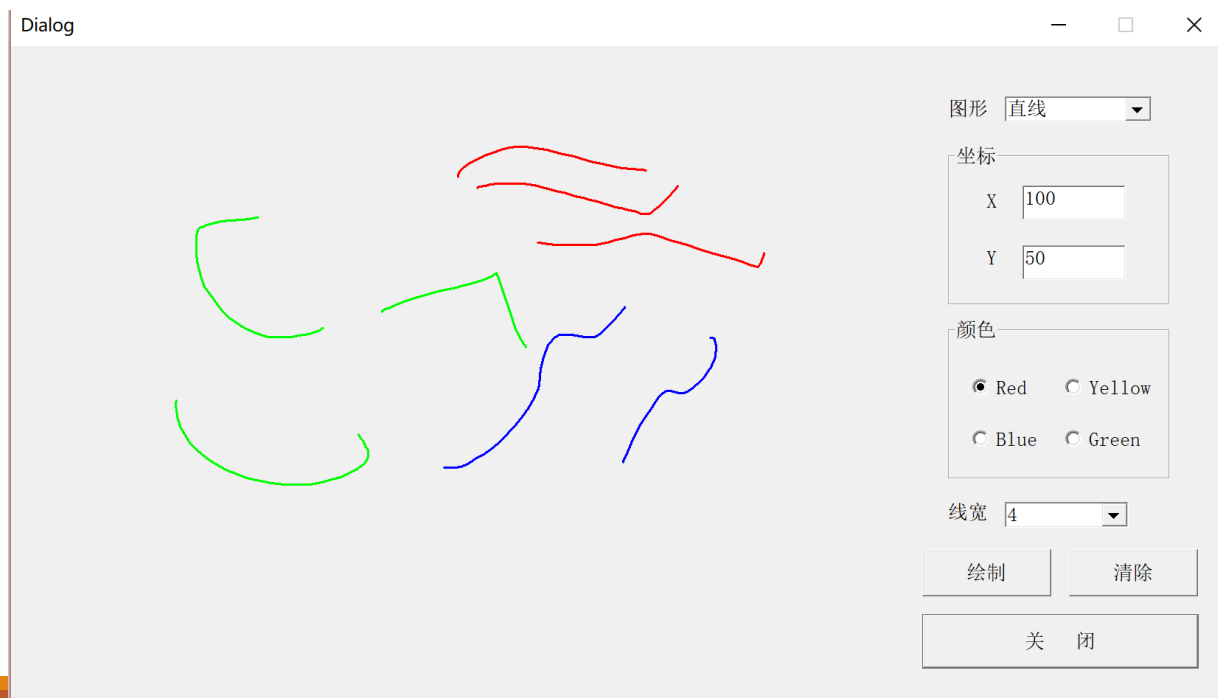
### 分析绘制鼠标移动轨迹的过程：

- 按下鼠标左键----开始记录线段开始位置
- 鼠标移动----把鼠标的当前位置作为上个直线终点和下个直线的起点
- 鼠标左键松开----停止画线段。

操作方式：在客户区中，按下鼠标左键，并移动鼠标绘制直线，左键弹起结束绘图。

注意几点：

- ✓ 直线包括两点：首点（动态）和末点（动态）；
- ✓ 当末点发生变化时，将新的起点和新的末点连线，调用 `Invalidate()` 重绘窗口。



事件的添加方法

窗口重绘

## 1. 为对话框类添加成员变量:

```
CPoint  m_ptOrigin; // 起始点坐标  
bool    m_bDragging; // 拖拽标记  
HCURSOR m_hCross; // 光标句柄
```

## 2. 在对话框类OnInitDialog()函数中设置拖拽标记和十字光标。在OnInitDialog()函数中添加代码:

```
// 初始化拖拽标记
```

```
m_bDragging=false;
```

```
// 获得十字光标句柄
```

```
m_hCross=AfxGetApp()->LoadStandardCursor(IDC_CROSS);
```

3. 为对话框类添加按下鼠标左键`WM_LBUTTONDOWN`的消息处理函数`OnLButtonDown(UINT nFlags, CPoint point)`, 添加下面的代码:

```
void CDrawDlg::OnLButtonDown(UINT nFlags, CPoint point)
{
    // TODO: 在此添加消息处理程序代码和/或调用默认值
    SetCapture(); // 捕捉鼠标
    SetCursor(m_hCross); // 设置十字光标
    m_ptOrigin = point;
    m_bDragging = TRUE; // 设置拖拽标记
    mousepointlist.clear();
    CDialogEx::OnLButtonDown(nFlags, point);
}
```

4. 为对话框类添加鼠标移动WM\_MOUSEMOVE的消息处理函数。

```
void CDrawDlg::OnMouseMove(UINT nFlags, CPoint point){
    if (m_bDragging)
    {
        CClientDC dc(this);
        UpdateData(TRUE);
        CPen PenNew;
        PenNew.CreatePen(PS_DASH, m_ilineWide, RGB(m_r, m_g, m_b));
        CPen* pPenOld = dc.SelectObject(&PenNew);
        ColorStruct col = { m_r, m_g, m_b };

        mousepointlist.push_back(point);
        dc.MoveTo(m_ptOrigin);
        dc.LineTo(point); // 绘制线段
        m_ptOrigin = point; // 新的起始点
        dc.SelectObject(pPenOld); // 恢复设备环境DC中原来的画笔
        PenNew.DeleteObject(); // 删除底层的GDI对象
    }
    CDialogEx::OnMouseMove(nFlags, point);
}
```

## 5. 为对话框类添加左键释放WM\_LBUTTONDOWN的消息处理函数。

```
void CDrawDlg::OnLButtonUp(UINT nFlags, CPoint point)
{
    // TODO: 在此添加消息处理程序代码和/或调用默认值
    if (m_bDragging)
    {
        mousepointlist.push_back(point);
        shapelist.push_back(Shapesel(self, mousepointlist, m_iline, m_iline,
            ColorStruct{ m_r, m_g, m_b }, m_index_type));

        m_bDragging = false; // 清除拖拽标记
        ReleaseCapture(); // 释放鼠标，还原鼠标形状
    }
    CDialogEx::OnLButtonUp(nFlags, point);
}
```

系统中任一时刻只有当前窗口才能捕获鼠标。在程序中需要时通过调用函数CWnd::SetCapture()捕获鼠标，使用鼠标画图结束后应该调用函数ReleaseCapture()释放鼠标。

# 流程图

响应MouseDown事件

鼠标左键按下

记录新的起点

绘图按钮是否按下?  
鼠标是否被按下?

否

是

记录新的末点, 绘制新线,  
记录新起点

鼠标左键是否弹起?

否

是

结束绘制

鼠标移动, 响应MouseMove事件  
是否绘制图形?

响应MouseUp事件

# 定时器的使用

---

- ◆ 添加**WM\_TIMER**消息处理函数，添加代码：

```
x += 10;  
Invalidate();
```

- ◆ 启动/停止定时器：

```
SetTimer(1,200,NULL);  
KillTimer(1);
```

- ◆ 在OnPaint()添加代码：

```
//CPaintDC dc(this);  
dc.TextOutA(x, 20, "timer test");
```

# 任务二

---

设计一个简单的图形绘制程序，要求具有以下功能：

- 1) 用鼠标拖动实现绘制直线段、椭圆、矩形；
- 2) 绘制的图形能够实现重绘；
- 3) 利用定时器实现动画绘制或计数器功能。
- 4) 扩展：绘制的图形保存到文件，打开保存的图形数据文件，并显示屏幕上。

# 文件读写

---

见补充PPT文件--- C++的IO流库

*The End*